Peer-to-Peer Offline Decentralized Mesh Network

PROJECT REPORT [Phase-I]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



Submitted by

Ayush Shukla (24U020062, Semester - V)

Yajur Chatnani (24U020056, Semester - V)

Under the supervision of

Dr. Sonal Telang Chandel
Assistant Professor (CSE)

11/2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



CERTIFICATE

This is to certify that the work embodied in this report entitled **"Peer-to-Peer Offline Decentralized Mesh Network"** has been satisfactorily completed by **Ayush Shukla (24U020062) and Yajur Chatnani (24U020056)** It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Engineering during the academic year 2026-27.

Date:

Name of Supervisor(s),

Dr. Sonal Telang Chandel

Assistant Professor,
Department of CSE
IIIT Bhopal (M.P.)

Head/Department Coordinator,

Dr. Rahul Chaurasia

Department Coordinator CSE
IIIT Bhopal (M.P.)



INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

We hereby declare that the following major project synopsis entitled “Peer-to-peer Offline Decentralized Mesh Network” presented in the is the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in** Computer Science and Engineering. It is an authentic documentation of our original work carried out under the able guidance of **Dr. Sonal Telang Chandel**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Ayush Shukla (24U020062) Sign of student1

Yajur Chatnani (24U020056) Sign of student2

ACKNOWLEDGEMENT

We, Ayush Shukla and Yajur Chatnani, would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our minor project titled "Peer-to-Peer Offline Decentralized Mesh Network".

First and foremost, we would like to extend our heartfelt thanks to our esteemed **Director**, Dr. Ashutosh Kumar Singh, for providing us with the necessary resources and facilities for conducting our project. His/her encouragement and vision played a key role in helping us accomplish our goals.

We are deeply grateful to our **Project Guide**, Dr. Sonal Telang Chandel, for his/her constant guidance, invaluable suggestions, and expert advice throughout the project. His/her knowledge and insights helped us enhance the quality of our work, and his/her support kept us motivated during difficult times.

Our sincere thanks to the **Department Coordinator**, Dr. Rahul Chaurasia, for the valuable support and encouragement provided during completion of the project.

We would also like to express our gratitude to all the **faculties** of the Computer Science and Engineering department for their valuable guidance and support in clarifying doubts and providing us with the right direction throughout the smooth execution of our project.

A special thanks to our **friends** for their continuous support, valuable suggestions, and helping us during various stages of the project. Their insights and encouragement motivated us to keep working on the project with enthusiasm.

Last but certainly not the least, we would like to express our deepest gratitude to our **parents** and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind our efforts.

We are truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

Ayush Shukla
24U020062
Yajur Chatnani
24U020056

AREA OF WORK

Our project will mainly focus on Mobile Ad-Hoc Networks (MANETs) and Delay Tolerant Networking (DTN) architectures using network simulation tools to develop off-grid emergency communication system. While conventional routing protocols can reliably create a mesh network, but the real challenge occurs in post-disaster scenarios where centralized communication infrastructure fails and there constant link breaks due to node mobility. It has been a fairly long enough period of time since wireless ad-hoc routing were introduced to communicate through a mesh network, but the deployment of low-power embedded devices in disaster zones poses a challenge for real-time data delivery. With time, we have acquired the capability to simulate complex topologies and overlays to bypass these physical limitations. In our project, we will be looking at the comparative performance analysis of this architecture across different hardware constraints (Raspberry Pi vs. ESP32) using the Network Simulator 3 (NS-3.48) framework [1] [2].

TABLE OF CONTENT

S.no	Title	Page No.
	Certificate	**2**
	Declaration	**3**
	Abstract	**9**
1	Introduction	**10**
2	Literature review	**15**
3	Problem Definition and Objectives	**19**
3	Methodology & Work Description	**21**
4	Proposed algorithm	**32**
5	Proposed flowchart/ DFD/ Block Diagram	**35**
6	Implementation (Tools & Technology Used)	**40**
7	Result Discussion and Analysis	**53**
8	Conclusion & Future Scope	**62**
9	References	**64**

LIST OF FIGURES

Fig	Description	Page no.
1.1	Mind map: Ad-Hoc Mesh Routing and DTN Paradigms	**12**
4.1	Overall system architecture of the proposed mesh network	**21**
4.2	Mesh SOS Android application workflow	**22**
4.3	End-to-End SOS message communication and forwarding workflow	**23**
4.4	Diagrammatic representation of a radio dead zone	**25**
4.5	Protocol selection architecture	**27**
4.6	SOS Packet Delivery Decision Flow	**28**
4.7	Watchdog Blackhole Detection Flow	**30**
6.1	System Architecture Block Diagram	**35**
6.2	Context Diagram of emergency wireless mesh system	**36**
6.3	Subsystem Decomposition	**36**
6.4	DTN & Watchdog Internal Process	**37**
6.5	End-to-End Application Packet Processing Flowchart	**38**
6.6	Mesh SOS Android Application Workflow	**39**
7.1	Development and Execution Environment	**40**
8.1	Impact of Hardware Constraints (RPI vs. ESP32) and DTN / QoS overlays on packet delivery ratio	**54**
8.2	Comprehensive ablation study across all 8 combinations of DTN, Watchdog security, and adaptive QoS	**56**
8.3	Aggregate application throughput comparison (RPI vs. ESP32)	**57**
8.4	Average End-to-End Delay Comparison in Log Scale (ms)	**57**
8.5	Average PDR improvement over pure MANET baseline across all 9 protocols	**58**
8.6	Granular per-flow PDR heatmap across SOS Anycast and SAR coordination schemes	**59**
8.7	Victim SOS Anycast vs SAR Unicast Coordination PDR Comparison	**59**
8.8	Layer-3 routing control and overlay relay transmissions	**60**

LIST OF TABLES

Table No.	Description	Page No.
4.1	Message Reliability Mechanisms in Mesh SOS	24
4.2	Comparison of Communication Transports in Mesh SOS	24
4.3	Wireless Simulation and Hardware Parameter Configurations	26
4.4	Offered Traffic Load Parameters	31
7.1	Hardware Specifications & Simulation Parameter Mapping	41
7.2	Development Technologies Used in MeshSOS	44
8.1	Packet Delivery Ratio Comparison Across Protocols & Hardware	53
8.2	Network Performance Gains by Hardware Profile	54
8.3	RPI 8-Mode Orthogonal Ablation Matrix (PDR %)	55
8.4	ESP32 8-Mode Orthogonal Ablation Matrix (PDR %)	55

ABSTRACT

In an event of major disaster or infrastructure breakdown, centralized communication towers or wired communication networks become unavailable. In such scenario, deployment of off-grid wireless mesh networks is vital for search and rescue (SAR) activities. The project addresses the development of offline peer to peer communication system based on Android system using WiFi Aware (Neighbor Awareness Networking) for device discovery and communication along with deployment of simulated Mobile Ad-Hoc Network (MANET) using Network Simulator 3 (NS-3) and its performance evaluation. The simulation is engineered to facilitate autonomous and zero configuration deployment based on decentralized MAC to IPv4 hashing without using centralized DHCP servers. The simulation evaluates different protocols in order to facilitate robust communication in highly dynamic disaster zones. A Watchdog security mechanism is implemented in the simulation to detect “Blackhole” relay nodes. A mechanism for handling network partitions and coverage holes using Delay Tolerant Networking (DTN) based Binary Spray and Wait mechanism is also implemented in the simulation.

Source Code: https://github.com/AyushShuklaIIIT/mesh-sos-and-ns3

INTRODUCTION

Background and Operational Context

Immediately after a natural disaster such as a hurricane, tsunami, earthquake, flash flood, or a major industrial accident, pre-existing telecommunications infrastructure may become unavailable or unreliable. Destruction of cellular towers (4G/5G towers), power grid failures, or destruction of underground fiber optic cables leave disaster affected regions in a complete communication blackout [3]. Any surviving infrastructure faces extreme network congestion due to traffic from the general public.

In these disaster-affected scenarios, it becomes very difficult for search-and-rescue teams, emergency medical technicians and field officers to exchange GPS coordinates, transmit real-time emergency alerts, etc [3]. Also, the trapped or injured disaster victims and displaced citizens lose access to their typical communication networks, leaving them unable to send SOS messages, or signal their real-world GPS coordinates for the SAR teams. Under such conditions, it becomes very important to set up an off-grid, decentralized wireless mesh network for emergency survival and disaster recovery.

Mobile Ad-Hoc Networks (MANETs) and peer-to-peer wireless mesh networks allow emergency responders and civilians with smartphones or low-power Single-Board Computers (SBCs) to dynamically construct multi-hop communication networks without any pre-existing centralized access points [4]. In a decentralized ad-hoc mesh network, participating nodes act both as endpoint terminals (civilians sending SOS alert messages) and as a wireless router by dynamically discovering multi-hop paths to transmit emergency payloads across intermediate relays.

Wi-Fi Aware-Based Offline Communication

In our project, Wi-Fi Aware is used to connect nearby Android devices when internet connectivity is not available. It allows devices to discover each other and establish a connection without using a Wi-Fi router. After discovery, the application establishes a data path between the devices and uses TCP sockets to exchange SOS messages. If a message fails to send, the application can retry sending the message, and acknowledgements help track the delivery of the message. This forms the communication part of our project, while message forwarding between connected devices is handled by the mesh routing component. [5]

Ad-Hoc Mesh Routing & Disruption Tolerant Networking

Ad-hoc wireless networking has various paradigms to address mobile wireless communications:

1. **Layer-3 Reactive MANET Routing (e.g., AODV, DSR):** Reactive protocols discover paths purely on demand when a node wants to send a message. Reactive protocols keep RF bandwidth free when no messages are being sent. However, they introduce initial route acquisition delays and generate massive broadcast storms (RREQ floods) when active links break due to mobility [6] [7].
2. **Layer-3 Proactive MANET Routing (e.g., OLSR, DSDV):** Proactive protocols continuously maintain routing tables over the entire network topology using periodic background control messages. It ensures no delay in the initial route setup to send an SOS packet, but it requires power to be used all the time. But when nodes are moving continuously, the previously computed routing tables quickly become stale. [8] [9].
3. **Layer-2 Hardware Mesh Protocols (e.g., IEEE 802.11s HWMP):** This protocol is a Layer-2 protocol and operates at the MAC layer and exchanges messages between points using the Airtime Link Metric. It is very efficient for static mesh networks, but it relies on 3-way handshake of Peer Management Protocol (PMP) which don't converge easily when nodes are moving continuously [10].
4. **Position-based and Hybrid Protocols (e.g., ZRP, GRP/GPSR):** Position-based protocols use physical coordinates for routing packets instead of global tables, and require scalable and decentralized location services that can independently maintain and resolve physical coordinates across the mesh [11]. Meanwhile, hybrid schemes combine local proactive zones with inter-zone reactive routing. [12] [13]
5. **Delay and Disruption-Tolerant Networking (DTN):** When physical network coverage gaps separate disaster victims and SAR teams into isolated clusters, conventional routing protocols fail completely and drop all non-routable packets. DTN overlays overcome network partitioning by introducing a Store-Carry-and-Forward architecture. Nodes buffer undelivered messages in local storage, physically carrying them as they move across terrain and deliver them on encountering any peers [14].
6. **Cross-Layer Security Mechanisms:** Disaster environments are vulnerable to compromised or malicious nodes dropping all received packets (Blackhole attack). Our simulation implements a Watchdog mechanism that uses dynamic trust scoring to isolate "Blackhole" relays [15].
7. **Autonomous Zero Configuration:** Deploying a centralized DHCP server in a disaster is impossible, which leads to discovering other mechanisms with which to identify a device in a mesh network [16]. So, our network overlay utilizes decentralized MAC-to-IPv4 FNV-1a hashing to autonomously assign IP addresses to endpoint devices.

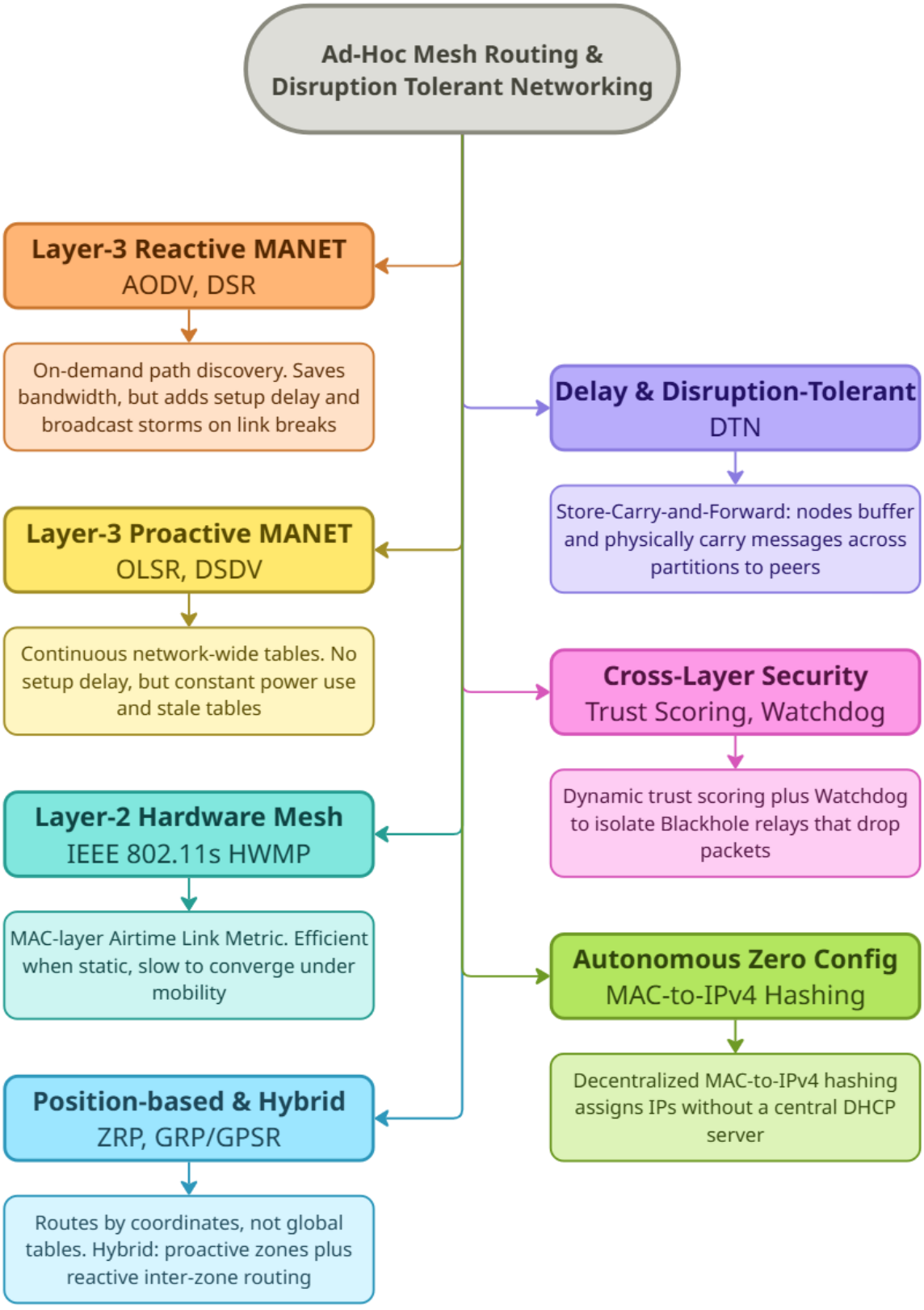


Figure 1.1: Mind Map: Ad-Hoc Mesh Routing & DTN Paradigms

Scope and High-Level System Overview

This project focuses on the development of an Android-based offline peer-to-peer communication system using Wi-Fi Aware, alongside the design and performance evaluation of a multi-protocol MANET and DTN framework simulated using Network Simulator 3 (NS-3.48) [1].

The simulation models a 500 m x 500 m disaster recovery zone populated by 45 mobile nodes following a heterogeneous mobility framework [17]:

- 12 Trapped Victims: Low-mobility nodes (0.05 – 0.3 m/s) simulating trapped survivors broadcasting SOS anycast messages.
- 18 Civilian Relays: Moving at pedestrian speeds (0.5 – 1.5 m/s) with 10s pause time to simulate bystander ad-hoc relays.
- 10 SAR Foot Squads: On-foot search and rescue teams moving at fast pedestrian speeds (0.8 – 1.8 m/s) with 5s pause times.
- 5 SAR Vehicles: Moving at vehicle speeds (5.0 – 12.0 m/s) with 2s pause times.

The physical layer is calibrated to simulate real-world responder and civilian SOS hardware (Raspberry Pi / ESP32 [18] / low-power Wi-Fi transceivers) using the IEEE 802.11g standard using a Log-Distance Propagation Loss Model and a transmission radius cap according to the hardware being used.

The simulation engine implements many routing protocols over different modes (AODV, OLSR, DSDV, DSR, HWMP, ZRP, GPSR, MULTIPATH, HYBRID) paired with a Binary Spray and Wait DTN fallback mechanism on UDP port 9999 [19].

The organisation of this project report involves discussing the following nine chapters that cover various aspects of the proposed Peer-to-Peer Offline Decentralized Mesh Network (MeshSOS). Chapter 1 provides general introduction about the project where emphasis is laid on the significance of communication during disasters and emergencies. It also highlights the problems and constraints in the existing communication infrastructure along with the requirements for decentralised peer-to-peer mesh network. Chapter 2 includes discussion on various technologies and research work done in wireless ad hoc network, decentralised communication, Wi-Fi Aware, Bluetooth communication, routing protocol, and Delay-Tolerant Networking (DTN) [20]. Chapter 3 discusses the problem statement and the problems faced by the project which relate to difficulties in sustaining communication without the use of normal network infrastructure. The aim and objectives of the project, as well as scope of the system are included in this chapter. Chapter 4 discusses the proposed methodology for the project where system architecture, network model, device discovery, connection between the devices, message forwarding and NS-3 [1] simulation methodology are covered in detail. Chapter 5 discusses algorithms used for the communication and message forwarding in the decentralised manner. The algorithms include multi-mode forwarding, queue processing, Binary Spray and Wait, Store-Carry-Forwarding and many others which were used for improving message

forwarding in intermittent networks. Chapter 6 provides the flowchart and block diagram of the system where the architecture of the mesh network, message communication, message routing mechanism and flowchart for the MeshSOS Android application are covered. Chapter 7 discusses the tools and technologies used for developing the project. It includes Android application development using Kotlin programming language, Wi-Fi Aware and Bluetooth classic communication, TCP sockets for data transfer, message storage in the local database and NS-3 [1] simulation. Chapter 8 presents the results of implementation and network simulations. It includes the performance of various routing and forwarding algorithms using certain evaluation metrics. Chapter 9 presents the summary of the major contribution of the project. It includes discussion on current limitations and possible improvement in future which include advanced mesh routing and many more.

LITERATURE REVIEW

Reactive (On-Demand) Protocols

Reactive routing protocols initiate route discovery only when a packet needs a transmission path.

- Ad-Hoc On-Demand Distance Vector (AODV): Developed by Perkins, Belding-Royer and Das, AODV uses Route Request (RREQ) and Route Reply (RREP) control frames to establish hop-by-hop forwarding path on demand. AODV uses destination sequence numbers to ensure that there are no loops and that the "count to infinity" problem does not occur. Nodes periodically send (broadcast) a HELLO message to keep the active link. Route Error (RERR) is generated if broken links seen - to invalidate stale entries. AODV reduces background traffic when not in use. But for mobile topologies in disaster situations, there are massive broadcast storms of RREQs, and initial route discovery delays until packets are assembled in local queues [6].
- Dynamic Source Routing (DSR): DSR is a stateless beacon-less routing protocol proposed by Johnson and Maltz. A network does not store all the intermediate hop addresses at intermediate nodes, but rather stores all those addresses in the packet header. Passive listening and aggressive route caching are crucial part of DSR. Beaconing is completely removed and overhead reduced in DSR. As network diameter or number of intermediate hops along the path increases, however, it can result in severe packet header bloat. The entire path must be carried in the packet header itself, so there is a reduced amount of effective throughput. Additionally, high packet loss and route salvaging delay under high node movement are caused by stale route caches. [7].

Proactive (Table-Driven) Protocols

Proactive protocols constantly maintain routing tables for all network destinations regardless of the traffic demand.

- Optimized Link State Routing (OLSR): a proactive link-state routing protocol standardized by Clausen and Jacquet in RFC 3626, which is suitable for mobile ad-hoc networks. Multipoint relay mechanism (MPR) is the important and novel feature in the OLSR protocol. A minimal number of 1-hop neighbours are chosen at each node to cover all 2-hop neighbours. A few selected MPR nodes relay the Topology Control (TC) message, avoiding significant amount of broadcast flooding overhead in comparison to the classical link-state algorithms. OLSR gives zero delay for delay sensitive streams since routes are precomputed. But these constant HELLO and TC messages take up a significant amount of RF bandwidth and battery power. OSLR has a convergence lag of about 3-5 seconds in case of high speed of events during disaster, when the packets are forwarded on outdated links that may no longer existt [8].

IEEE 802.11s Hybrid Wireless Mesh Protocol (HWMP)

HWMP is a path selection protocol as specified in the IEEE 802.11s that is required in wireless mesh networks as the default path selection protocol [21]. HWMP is a hybrid approach of both proactive tree-based mode using Root Announcement (RANN) frames and reactive mode that is based on ad hoc on-demand distance vector (AODV) using Path Request (PREQ) frames and Path Reply (REP) frames.

HWMP evaluates wireless path quality using the Airtime Link Metric (ALM), which calculates channel time consumed to transmit a standard test frame:

$$m = \left(O_v + O_p + \frac{B_s}{r}\right)\frac{1}{1 - e_f}$$

where O_v is the MAC overhead, O_p is the preamble duration, B_s is the frame size in bits, r is the PHY bit rate, and e_f is the frame error rate. HWMP offers auto-configuration and MAC-Layer bridging. But empirical disaster studies show that HWMP is created specifically for static mesh networks. It is based on 3-way handshakes of stateful Peer Management Protocol (PMP). If the nodes are moving continuously then, peer-link establishment fails and results in packet drops. [22].

Wi-Fi Aware (Neighbour Awareness Networking)

Supported Android devices can find other devices that have Wi-Fi Aware enabled without using a Wi-Fi router. This can be helpful when you don't have internet access like it would during a disaster. It is utilized in our project in order to find out which devices are nearby and create a direct connection for exchanging SOS messages. Once a connection is made the application sends these messages via TCP sockets. But just two devices does not necessarily form a multi-hop mesh network. For devices to communicate with each other via message forwarding, there needs to be an additional routing system. [5]

Android-Based Offline Communication

The primary problem for MeshSOS isn't just getting a message from one Android phone to another. The application should deal with the case of no connection or no message acknowledgement. WiFi Aware is a method used to find devices in the same vicinity, and to create a communications channel. After connecting the phones are able to send an SOS message over TCP sockets. An alternative transport is available, i.e., Bluetooth Classic. [23] The delivery and routing of messages are separate functions. Acknowledgments are used to track delivery in the application, and the application will retry messages that were not delivered. This mesh routing allows for forwarding messages to other peers and queuing messages for delivery when peers reconnect. This is significant because if one phone is connected to another, it does not necessarily mean that delivery is possible over a multi-hop network.

Multi-Hop Message Forwarding

With MeshSOS, a message can be forwarded to other phones if the sender and receiver are not directly connected. Once a message is received on a phone, it can relay the message to other connected phones, thus extending the message through the network. In the application, the MeshRouter takes care of it. It stores the received SOS calls and determines if calls have been received already before passing them on. It also prevents the phone that it was sent from from receiving a message back. The same routing component is responsible for acknowledgements and also passes them to its peer nodes. This is the component that enables MeshSOS to go beyond exchanging messages among two phones. The difference between Store-Carry-and-Forward and the other types of telegraphic services is that the messages cannot be sent at the time, but must be stored for later delivery. [20]

Smartphone-Based MANETs and Decentralized Networks

Typically, in a normal wireless network, devices communicate via a router or an AP. In contrast, Mobile Ad Hoc Networks (MANET) operate in a unique way, as devices are able to communicate directly with each other and some devices can relay messages to others. [24] As an example, if two phones are too far away from each other to communicate directly, they might be able to communicate via another phone. This will depend on the connections available and the routing protocol. In the event of a disaster, these links can be unstable. People can move, phones can be out of range and a path that used to work might not work anymore. If messages are to reach their destination the network must be able to cope with such changes.

This is relevant to MeshSOS because sending an SOS message to a nearby phone is only the first step. The message may need to pass through other devices to reach someone farther away. The Android application handles communication between connected devices, while the NS-3 simulation is used to study how different routing protocols perform under changing network conditions.

Store-Carry-and-Forward Architecture

In disaster scenarios, physical obstacles and large distances between nodes partition the network into subgraphs where continuous end-to-end paths do not exist between two nodes [19]. Traditional MANET routing protocols immediately fail in such a scenario, dropping all non-routable packets.

Delay and Disruption-Tolerant Networking (DTN), defined by Fall, Cerf and the DTRNG, addresses this challenge by introducing a Bunder Layer [19] above the Transport Layer. DTN replaces real-time end-to-end forwarding with asynchronous Store-Carry-and-Forward architecture.

1. Store: When an active path is unavailable, bundles are stored in non-volatile memory.
2. Carry: Mobile nodes physically carry the stored bundles across network gaps.

3. Forward: When the node encounters another node or reaches the destination, it transfers the bundle over the wireless link [19].

Store-and-forward communication is considered in both the Android application and the NS-3 simulation in both ways, but in different ways. In MeshSOS, messages are locally stored and can be sent to connected peers when they are available. The NS-3 simulation incorporates Delay-Tolerant Networking separately with the Binary Spray-and-Wait approach to study message delivery when network connectivity is interrupted when network connections are interrupted. This allows the project to investigate both the application-Level message processing and DTN forwarding behavior in a simulated network, as well as in both application-level message handling and DNC forwarding.

DTN Replication Strategies and Routing Protocols

1. Epidemic Routing: This flooding scheme replicates bundles to every node it meets that does not already hold a copy of the bundle. While it achieves minimum latency, it causes severe memory drain and buffer blockage. [25].
2. Binary Spray & Wait: A ticket-bounded protocol that limits bundle replication to L copies.
    a. Spray Phase: The source node assigns L tickets to a bundle. When a node holding $n > 1$ tickets encounters a peer, it transfers $\lfloor n/2 \rfloor$ tickets to the relay and keeps $\lceil n/2 \rceil$ tickets.
    b. Wait Phase: Once a node's ticket count drops to $n=1$, it stops replicating and delivers exclusively when it physically reaches the destination [14].
3. PROPHET (Probabilistic Routing Protocol using History of Encounters and Transitivity): Tracks node encounter histories and calculates probability of transmission. Forwards only bundles to nodes with a higher probability of reaching the target destination [26].
4. Single-Copy Schemes:
    a. Direct Delivery: Holds the bundle until it encounters the destination directly. It incurs zero application overhead but also leads to unbounded delay.
    b. First Contact: It forwards the single copy to the first peer it encounters leading to a random-walk search.

PROBLEM DEFINITION AND OBJECTIVES

Just after a severe disaster, centralized communication services (like power grids, cell towers, fiber optical cables, etc.) usually break or become congested. This creates two critical communication problems:

1. Civilian victims are unable to send SOS messages or GPS coordinates to emergency services or Search and Rescue (SAR) teams.
2. First responders lose voice, text, and coordination within their ranks.

While Mobile Ad-Hoc Networks (MANETs) create peer-to-peer topologies to send messages, deploying standard ad-hoc mesh protocols in a disaster environment introduces several bottlenecks:

- End-to-End Path Assumption: Protocols like AODV, OLSR, DSDV, DSR, IEEE 802.11s HWMP require end-to-end continuous paths. Physical obstructions and node mobility create isolated network subgraphs, causing conventional mesh protocols to drop 100% of their non-routable SOS packets.
- Control Message Storms & Mobility Failures: Node movement triggers RREQ floods and TC updates that flood the RF spectrum and drain device's batteries.
- Malicious Interception: Decentralized networks are vulnerable to blackhole nodes that silently drop all packets they receive (Blackhole Attack).

Project Objectives:

The primary objective is to design, implement and benchmark a multi-protocol MANET and DTN simulation framework in NS-3.48 for civilian and SOS first responder communications in a disaster scenario [2].

- Disaster Topology & Mobility Framework: Model a 500m x 500m grid simulating a small disaster region with 45 mobile nodes: 12 Trapped Victims (0.05-0.3 m/s), 18 Civilian Relays (0.5-1.5 m/s), 10 SAR Foot Squads (0.8-1.8 m/s), and 5 SAR Command Vehicles (5.0-12.0 m/s)
- Realistic Hardware Wireless Stack: Configure IEEE 802.11g (12 Mbps Data Rate, 6 Mbps Control Rate, 20 dBm transmission power) with a Log Distance Path Loss Model (n = 2.5) with a maximum 200m transmission radius to reflect real-world Raspberry Pi transmission capabilities [27].
- Multi-Protocol Simulation Implementation: Develop a C++ script supporting multiple routing protocols and modes including AODV, OLSR, DSDV, DSR, HWMP, ZRP, GRP/GPSR, MULTIPATH and HYBRID with two modes: DTN overlay ON and OFF.
- DTN Overlay: Implement a custom overlay on UDP Port 9999 incorporating Binary Spray & Wait DTN, and adaptive 1-hop HELLO beacon to store and carry SOS

payloads across physical network gaps [14].

- Watchdog Security: Implement passive monitoring to calculate dynamic trust score (0.0 – 1.0) and isolate blackhole nodes.
- Autonomous Zero-Configuration (Zero-Conf) Architecture: Eliminate the reliance on centralized DHCP server to obtain a unique IP address for each device on the Mesh Network using a decentralized FNV-1a MAC-to-IPv4 hashing algorithm to automatically assign IP addresses.
- Distributed Location Discovery: To get the location of a node for GRP/GPSR routing, use Over-the-Air (OTA) location cache, forcing nodes to learn target coordinates strictly through piggybacked packets and 1-hop/2-hop neighbors' HELLO packets.
- Android-Based Offline Communication Application: Develop MeshSOS, an Android application that allows users to send SOS messages without internet access. The app uses Wi-Fi Aware to connect to nearby devices and Bluetooth Classic as an alternative. It also supports message forwarding between connected phones and keeps messages saved when they cannot be sent immediately.

PROPOSED METHODOLOGY AND WORK DESCRIPTION

1. Overall System Architecture and Network Model

To address the challenges of off-grid emergency communication, this project proposes a multitier ad-hoc, wireless mesh architecture designed specifically for disaster recovery zones. The system provides a communication framework which serves both disaster civilians distress signal broadcasting (SOS message) and tactical first responders coordination without relying on centralized communication architectures like power grids, cellular towers, fiber optic networks, etc.

The overall system architecture of the proposed mesh network is shown in Figure 4.1, illustrating the two main components: the MeshSOS Android Application and the NS-3.48 Network Simulation.

We have also developed an Android application called MeshSOS as part of this project. The main purpose of the app is to send SOS messages when internet is not available. It uses Wi-Fi Aware to find nearby phones and connect with them. Bluetooth Classic is also available as another option. We have also worked on forwarding messages between connected phones so that a message can reach beyond the phone that originally sent it.

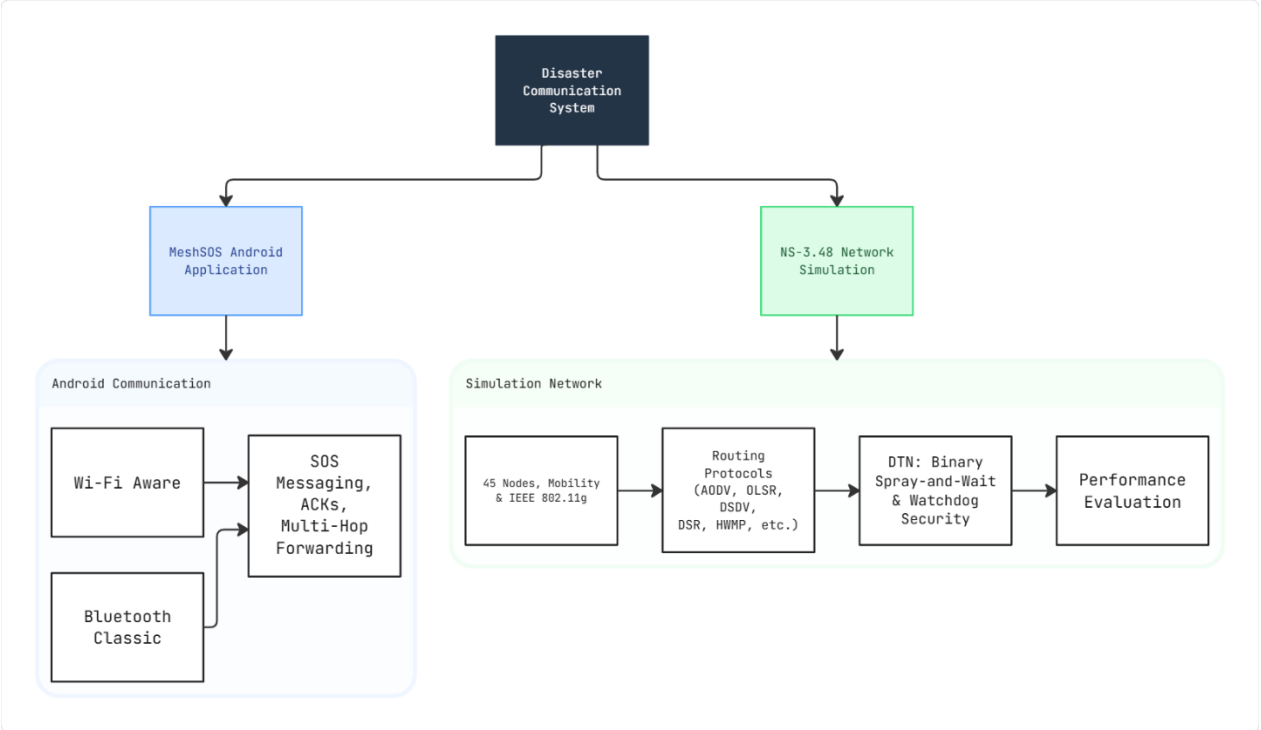


Figure 4.1: Overall System Architecture of the Proposed Mesh Network

The simulation network deploys a 500 m x 500 m spatial disaster area with 45 mobile nodes. Nodes are divided into four operational categories:

- Trapped Victims (Nodes 0-11): Represents low-mobility survivors generating SOS

messages.

- Civilian Relays (Nodes 12-29): Act as bystander ad-hoc relays.
- SAR Foot Squads (Nodes 30-39): Represents on-foot search and rescue teams.
- SAR Command Vehicles (Nodes 40-44): These nodes serve as high-speed mobile nodes.

2. Android Application Implementation (MeshSOS)

2.1 Device Discovery and Connection Establishment

In order to counter any problems regarding communication during an emergency situation where no electricity is present, a hierarchical structure has been proposed for this particular project, which makes use of an ad-hoc, wireless mesh technology in order to provide a communication infrastructure that helps to broadcast distress signals and coordinate the efforts of first responders without depending on centralized communication networks such as electrical grid, cell towers, fiber optics, etc. As part of this project, we have built an Android application named MeshSOS. Its main functionality is to broadcast SOS message in case there is no Internet connection available. This is done by using Wi-Fi Aware technology to discover nearby devices and connect to them. Bluetooth Classic technology is also available as another option. We also implemented message forwarding functionality among connected devices in order to extend message broadcast beyond original sender device.

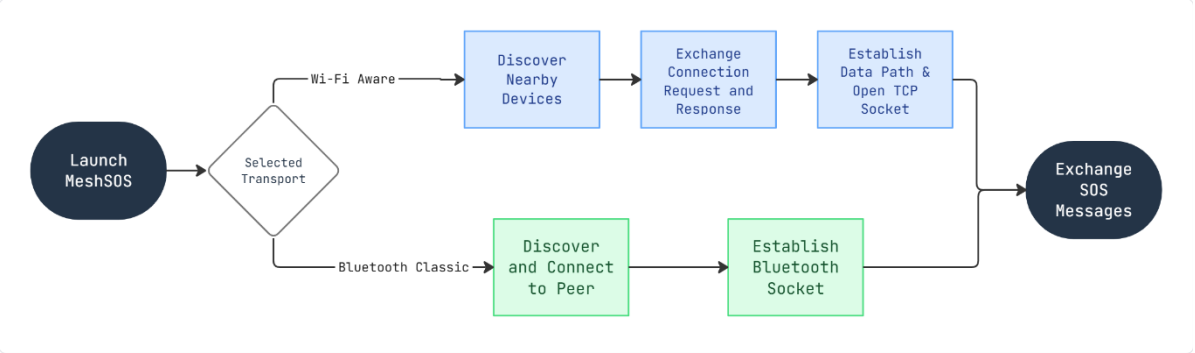


Figure 4.2: MeshSOS Android Application Workflow

2.2 SOS Message Routing and Multi-Hop Forwarding

The application has built in functionality that allows the SOS messages to be forwarded from one device to another. When a SOS message is generated by the user it goes through the application process and gets transmitted to the peers available. The message is then validated at the receiving end and if it has already been received it will not be processed.

If the SOS message received is considered valid then it can be forwarded to any other connected peer except the peer from whom it came from. This allows the message to

traverse through multiple devices rather than being stuck within the range of the sender device only. Acknowledgements (ACKs) are also possible using this application.

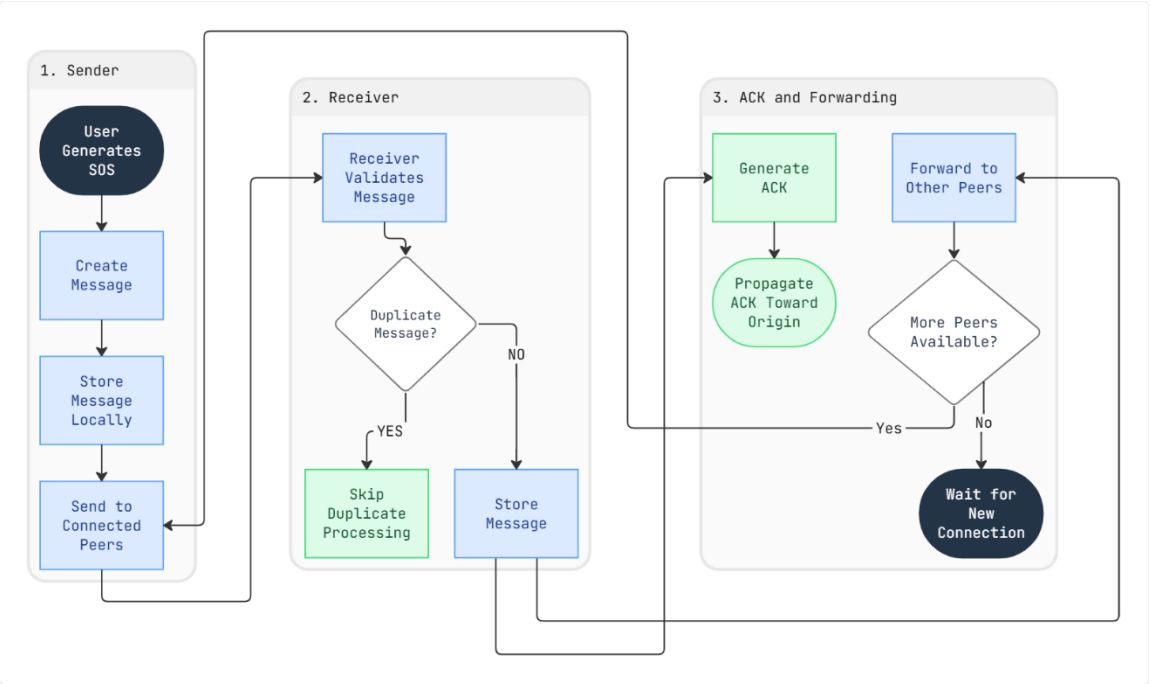


Figure 4.3: End-to-End SOS Message Communication and Forwarding Workflow

The forwarding technique makes use of message identifiers and duplication avoidance in order to minimize redundancy and unnecessary re-transmissions. There is also a restriction on the hop limit and message expiration time after which the message cannot remain in circulation. If the message is received by a machine that has already processed the message before, it will not process it again.

2.3 Message Reliability and Offline Persistence

Considering that the devices in a disaster area can frequently go out of reach, the message will not necessarily be delivered immediately. This problem is solved by MeshSOS using local storage of messages and attempting to deliver the message when appropriate peers become available.

The use of the Room database ensures that there is local storage for the messages. The messages waiting to be delivered will keep trying to be delivered with increasing time intervals until the retry limit is reached. If the device connects to the network, the messages that were stored in the database can be delivered to the appropriate peers, as long as the message has not yet expired and/or has not been delivered.

The acknowledgement feature ensures that the application knows if the SOS message has reached the correct state.

The main reliability mechanisms are summarized in Table 4.1.

Table 4.1: Message Reliability Mechanisms in MeshSOS

Mechanism	Purpose
Duplicate detection	Prevents repeated processing of the same message
Hop limit and expiry	Restricts message propagation and prevents indefinite circulation
Local message storage	Maintains messages across temporary disconnections
Message acknowledgement	Confirms receipt and propagates acknowledgement information
Retry mechanism	Attempts delivery again when an earlier attempt is unsuccessful
Store-and-forward	Allows messages to be forwarded when peers become available

2.4 Communication Transport Comparison

MeshSOS supports two communication options. Their main differences are summarized below in Table 4.2.

Table 4.2: Comparison of Communication Transports in MeshSOS

Feature	Wi-Fi Aware	Bluetooth Classic
Device discovery	Wi-Fi Aware service discovery	Bluetooth device discovery
Communication	TCP socket over an established Wi-Fi Aware data path	Bluetooth socket
Internet requirement	Not required	Not required
Intended role	Direct communication between compatible nearby devices	Alternative communication when Wi-Fi Aware is unavailable or not selected

Current connection handling	Supports multiple peer connections, subject to implementation limits	Current implementation maintains one active peer connection

Wi-Fi Aware acts as the main transport method for multipoint communication, whereas Bluetooth Classic serves as an alternative option to establish a connection between two devices. The number of simultaneous peers and the range of connection depends upon the implementation and the devices that are available.

3. Heterogeneous Disaster Mobility Framework

Standard ad-hoc simulation studies usually utilize uniform mobility nodes where each node moves at identical speeds. However, real-world disaster scenarios exhibit extreme mobility variance (such as the coverage gaps shown in Figure 4.4). To reflect this reality, a Heterogeneous Mobility Framework is implemented using RandomWaypointMobilityModel and RandomRectanglePositionAllocator in NS-3.48 [17].

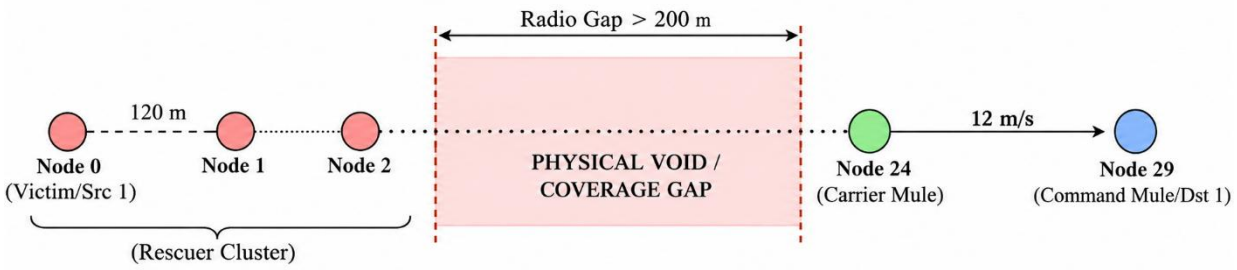


Figure 4.4: Diagrammatic representation of a radio-dead zone

3.1 Networking Impact of Heterogeneous Mobility

- Cluster Formation: Slow pedestrian speeds cause ground units to form localized, semi-stable subgraphs.
- Temporary Disconnections: As pedestrian nodes pause to conduct search operations, physical separation between clusters exceeds the radio range, causing Layer-3 route invalidation.
- Delay-Tolerant Networking: Fast travelling nodes are able to traverse the distance between network clusters, collecting buffered Delay-Tolerant Networking (DTN) bundles and delivering them on coming in 1-hop distance of the destination.
- Over-the-Air Location Cache Staleness: Since nodes learn physical locations via

decentralized Over-the-Air (OTA) gossiping. High-speed mobility nodes (SAR command vehicles moving at up to 12.0 m/s) quickly invalidate cached geographic coordinates, requiring continuous 1-hop and 2-hop HELLO beacon updates to prevent routing failures during greedy forwarding.

4. Physical Layer, Transceiver Hardware Simulation & Wireless Channel Modeling

To ensure simulation results translate to real-world embedded hardware (such as Raspberry Pi single-board computers), the physical channel stack and channel models are calibrated to match real-world RF specifications as described in Table 4.3:

Table 4.3: Wireless Simulation and Hardware Parameter Configurations

Physical Parameter	Simulation Parameter Value	Hardware & Environmental Justification
Wi-Fi Standard	IEEE 802.11g (2.4 GHz ISM)	Superior wall penetration & diffraction as compared to 5 GHz 802.11ac/ax.
Data Bitrate	12 Mbps (ErpOfdmRate12Mbps)	QPSK 1/2 modulation; provides high receiver sensitivity and high immunity.
Control Bitrate	6 Mbps (ErpOfdmRate6Mbps)	Mandatory Basic Rate; BPSK 1/2 modulation ensures maximum range for beacons.
Transmit Power	20.0 dBm (100 mW)	Aligns with Raspberry Pi CMOS Power Amplifier output limits and ETSI EIRP caps [27].
Antenna Gain	3.0 dBi (Tx and Rx)	Models standard omnidirectional dipole antennas.
Path Loss Model	Log-Distance (n = 2.5)	Represents semi-open damaged urban environments with rubble and foliage.
Max Physical Range	200.0 meters	Hard boundary cap enforced via RangePropagationLossModel.

4.1 Technical Derivation of Wireless Parameters

- Why 12 Mbps Data Rate (ErpOfdmRate12Mbps): While 802.11g supports data rate up to 54 Mbps (using 64-QAM 3/4), high-order modulation requires an extremely high Signal-to-Noise Ratio (SNR > 25 dB) and is prone to corruption under mobility. Setting data traffic to around 12 Mbps (QPSK 1/2) reduced the required SNR threshold (≈ 10 dB), keeping multi-hop links operational across moving nodes.
- Why 6 Mbps Control Rate (ErpOfdmRate6Mbps): Under IEEE 802.11 rules, all control frames (RTS/CTS, ACK) and routing/overlay beacons must be transmitted at a

mandatory basic rate readable by all nodes. 6 Mbps (BPSK 1/2) offers the maximum possible receiver sensitivity (≈ -93 dBm), mitigating hidden node collisions at the mesh boundaries.

- Log-Distance Propagation Loss Formulation: Signal attenuation across distance d is modeled as:

$$L(d) = L_0 + 10 \cdot n \cdot \log_{10}\left(\frac{d}{d_0}\right)$$

where d_0=1.0 m (reference distance), L_0=40.0 dB (reference loss for 2.4 GHz at 1 m), and n=2.5 (path loss exponent). Path loss doubling yields:

$$\Delta L = 10 \cdot 2.5 \cdot \log_{10}(2) \approx 7.52 \text{ dB loss per distance doubling}$$

- Range Boundary Cap: To prevent unrealistically long low-power decodes, RangePropagationLossModel enforces a 200.0 m hard cap. Transmissions beyond 200 m experience an immediate attenuation to -1000 dBm, dropping packet reception to zero.

5. Multi-Protocol Simulation Architecture

The C++ simulation engine provides a unified execution platform that benchmarks 11 different ad-hoc routing paradigms within the same physical and mobility framework. The architecture for this protocol selection is detailed in Figure 4.5.

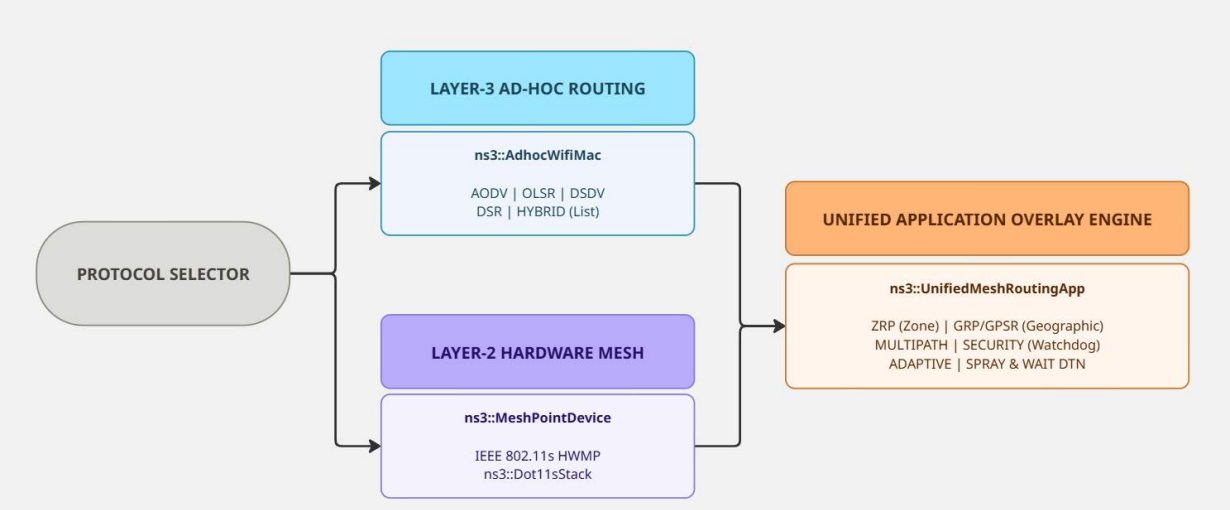


Figure 4.5: Protocol Selection Architecture

6. DTN Application Overlay

When network disconnections break Layer-2/Layer-3 paths, standard IP stacks drop all non-routable packets. To guarantee message delivery, the system introduces a custom application-layer overlay working on UDP Port 9999. The SOS packet delivery decision flow which this overlay follows is shown in Figure 4.6.

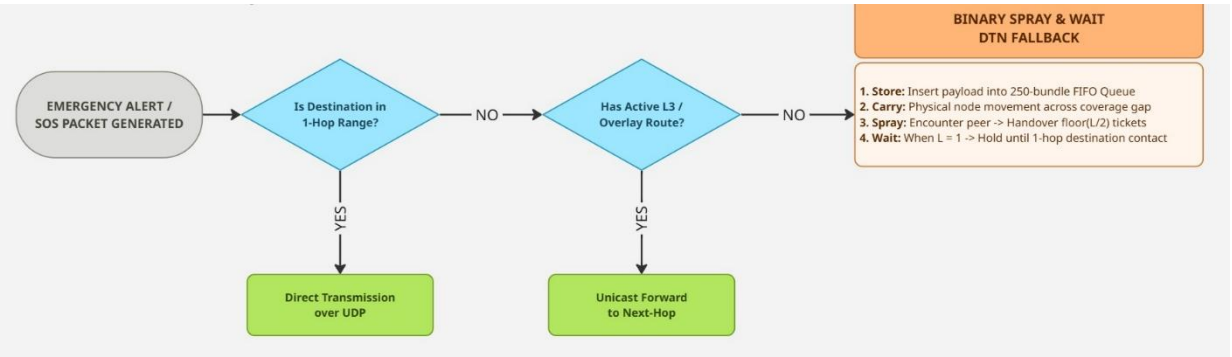


Figure 4.6: SOS Packet Delivery Decision Flow

6.1 Plain Proof-of-Delivery (POD) Wire Header

To ensure portable, zero-overhead byte serialization across NS-3 nodes, the overlay uses a 51-byte C++ structure:

```
struct OverlayWireHeader
{
    uint8_t pktType;          // Overlay packet type
    uint8_t senderRole;       // Disaster node role of immediate sender
    uint8_t survivorCount;    // Number of trapped survivors reported in Victim SOS (1..4)
    uint8_t gpsrMode;         // if greedy then 0, if perimeter then 1
    float failureDist;        // Distance to destination where packet entered Perimeter Mode
    uint32_t srcIp;           // Original Victim or SAR Sender IP
    uint32_t dstIp;           // 0.0.0.0 for PKT_SOS_ANYCAST; Target SAR IP for PKT_SAR_COORD
    uint32_t prevHopIp;       // Immediate Relay IP
    uint32_t seqNum;          // Unique Bundle Sequence ID
    uint64_t txTimeNs;        // Creation Timestamp (ns)
    uint16_t flowId;          // Flow ID (1..4 = Victim SOS Clusters, 5..6 = SAR
Coordination)
    uint16_t tickets;         // Binary Spray-and-Wait Tickets
    uint8_t ttl;              // Remaining Hop Count
    float originGpsX;         // Sender's own GPS X (from onboard GNSS receiver)
    float originGpsY;         // Sender's own GPS Y (from onboard GNSS receiver)
    float dstGpsX;            // Target's last known GPS X stamped in packet header
    float dstGpsY;            // Target's last known GPS Y stamped in packet header
    uint8_t hasValidDstGps;   // 1 if sender knew target's GPS via OTA Location Cache, 0
otherwise
    float batteryPct;         // Node Battery Percentage
    uint8_t neighborCnt;      // Advertised 1-Hop Neighbor Count (for ZRP 2-Hop Zone)
    uint32_t neighbors[6];    // Advertised 1-Hop Neighbor IPs
    uint32_t nearestSarIp;    // Known 1-Hop/Multi-Hop SAR Responder IP
    float nearestSarGpsX;     // Known SAR Responder GPS X
    float nearestSarGpsY;     // Known SAR Responder GPS Y
    uint32_t gossipSarIp;     // Rotating SAR Unit IP for Unicast SAR-to-SAR Coord
    float gossipSarGpsX;      // SAR Unit GPS X
    float gossipSarGpsY;      // SAR Unit GPS Y
    uint64_t gossipTimeNs;    // Timestamp when SAR GPS was recorded
};
```

6.2 Periodic Hello Beaconing & Topology Discovery

Every node executes a periodic timer (SendHelloBeacon()) that broadcasts a PKT_HELLO_BEACON frame to 255.255.255.255:9999:

- Jitter for Collision Avoidance: Initial beacons are scheduled with uniform random jitter (0.01s to 0.4s) to prevent MAC-layer broadcast storms.
- Neighbor Table Management: Receiving nodes populate an in-memory table containing neighbor Ips, physical coordinates, battery percentages, and 2-hop neighbor lists. Entries which are not heard for >3.0s are automatically removed from the table.
- Adaptive Timer: In ADAPTIVE mode, if neighbor density remains constant, the beacon interval dynamically increases (multiplied by 1.5) up to 3.0 seconds to save power; if any change is received from neighbor, then the timer is reset back to 0.5s to quickly converge.

6.3 Binary Spray & Wait DTN Ticket-Splitting Mechanism

When an emergency payload cannot be routed directly via Layer-3 or overlay paths, it enters the Binary Spray & Wait DTN Phase.

1. Ticket Initialization: Payloads are initialized with L=16 logical tickets.
2. Binary Spray Phase (L > 1): When a node holding n > 1 bundles encounters a node which does not have any copies:
    a. Tickets Sprayed to Relay = $\left\lfloor \frac{n}{2} \right\rfloor$
    b. Tickets Kept by Source = $\left\lceil \frac{n}{2} \right\rceil$
3. Wait Phase (L = 1): When a node's ticket count drops to 1, ticket replication stops. The node holds the bundle in its 250-bundle FIFO queue, physically carries it, and only transmits it when entering 1-hop distance of the destination.

7. Watchdog Security and Trust

Decentralized ad-hoc mesh networks in disaster zones are vulnerable to malicious nodes which silently drop emergency packets (Blackhole attacks). So the system incorporates a passive Watchdog Security System to detect and isolate malicious nodes as shown in Figure 4.7.

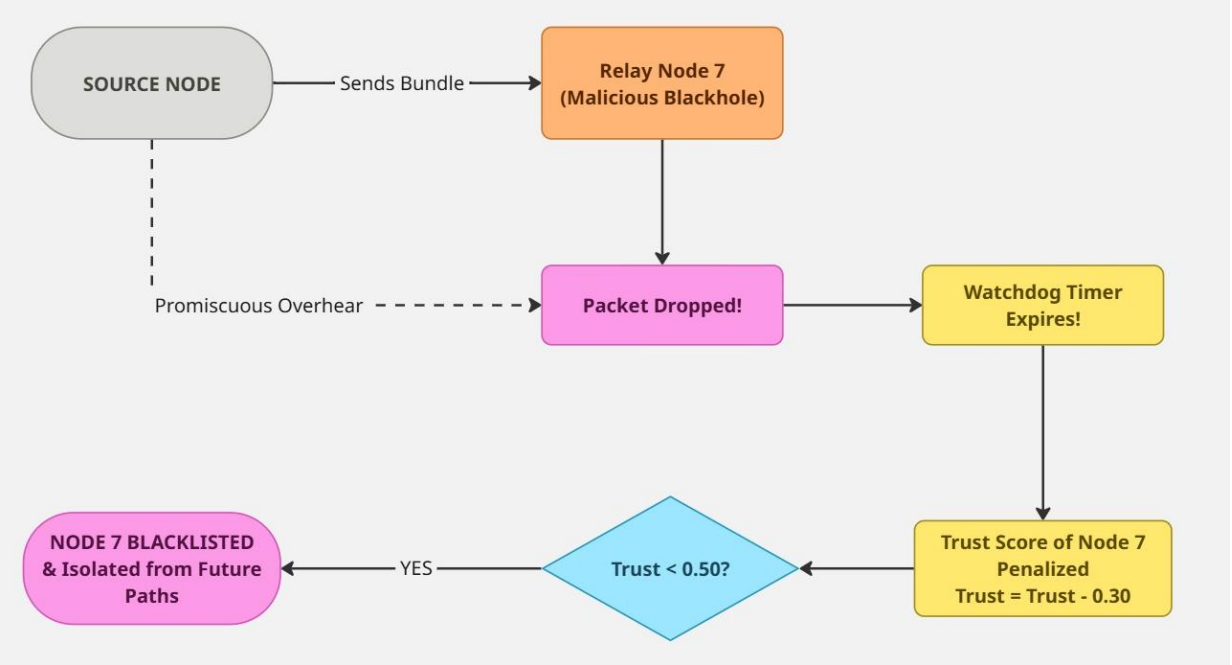


Figure 4.7: Watchdog Blackhole Detection Flow

Trust Score Calculation

- Malicious Node Designation: In SECURITY mode, Nodes 7, 14 and 21 are configured as malicious Blackhole nodes that accept incoming bundles but silently drop them without forwarding.
- Overhear Acknowledgements: As node A sends the packet to node B, node A enters its m_pendingWatchdog map for further processing and also sets the verification timer for 0.8 seconds.
- Trust Score Adjustments:
  - Successful Forward: If Node A overhears Node B transmitting the bundle withing 0.8s, Node B's trust score increases ($T_B = \min(1.0, T_B + 0.05)$)
  - Forwarding Failure: If the timer expires without overhearing retransmission, B's trust score is penalized ($T_B = \max(0.0, T_B - 0.30)$)
- Path Isolation: When selecting next-hop relays, nodes evaluate a security-weighted routing cost:

$$\text{Routing cost} = \frac{Distace\ to\ Destination}{\max(0.1, T_{relay})}$$

Nodes with $T_{relay} < 0.50$ are completely isolated from active routing paths.

8. Emergency Traffic Flow & FlowMonitor Telemetry

To evaluate system performance under realistic disaster communications, we have set up the parameters in the code as given in Table 4.4:

Table 4.4: Offered Traffic Load Parameters

Parameter	Value
Payload Size	1024 bytes per packet
Flow Volume	200 packets per flow (800 total application packets injected per run)
Transmission Interval	0.25 seconds between packets (4 packets / s)
Offered Rate Per Flow	1024 x 8 x 4 = 32,768 bps ≈ 32.77 Kbps
Aggregate Network Offered Load	32.77 x 4 ≈ 131.07 Kbps

End-to-End Packet Tracking & Metric Formula

The core performance metrics are calculated using the following formula:

1. Packet Delivery Ratio (PDR%):

$$\text{PDR} = \left(\frac{\sum_{i=1}^{N_{\text{flow}}} \text{RxPackets}_i}{\sum_{i=1}^{N_{\text{flow}}} \text{TxPackets}_i} \right) \times 100\%$$

2. Average End-to-End Latency (ms):

$$\text{Average Delay} = \frac{\sum_{j=1}^{\text{RxPackets}} \left(t_{\text{receive,j}} - t_{\text{transmit,j}} \right)}{\text{RxPackets}} \times 1000 \text{ ms}$$

3. Sustained Throughput (Kbps):

$$\text{Throughput} = \frac{\text{Total Received Payload Bits}}{\left(t_{\text{last_rx}} - t_{\text{first_tx}} \right) \times 1000} = \frac{\text{RxBytes} \times 8}{\left(t_{\text{last_rx}} - t_{\text{first_tx}} \right) \times 1000} \text{Kbps}$$

PROPOSED ALGORITHMS

Algorithm 1: Unified Queue Processing & Multi-Mode Forwarding

This algorithm represents the primary execution loop executed by every node every 0.25 seconds to process buffered payloads, evaluate routing metrics and trigger DTN fallback.

INPUT: Buffer Queue m_buffer, Neighbor Table m_neighborTable, Active Mode m_mode, DTN Flag m_enableDTN

OUTPUT: Forwarded Unicast Packets / Sprayed DTN bundles

```
Neighbors ← GetActiveBeaconNeighbors()
now ← Simulator::Now()
myPos ← GetMyPosition()

FOR EACH bundle B IN m_buffer DO
    // Drop expired bundles in pure non-DTN mode
    IF m_enableDtn = FALSE AND (now - B.creationTime) > 2.5 seconds THEN
        m_buffer.erase(B)
        CONTINUE
    END IF

    dstIdx ← GetNodeIndexByIp(B.destination)
    dstPos, hasOtaGps <- LookupOverTheAirLocationCache(B.destination)
    forwardedPrimary ← FALSE

    // Step 1: Direct 1-Hop Delivery Check
    IF dstIdx IS IN m_neighborTable THEN
        TransmitBundleTo(B, B.destination, ticketsToGive = 1)
        m_buffer.erase(B)
        CONTINUE
    END IF

    // Step 2: Layer-3 IP Routing Modes (AODV, OLSR, DSDV, HYBRID)
    IF m_mode IN {"AODV", "OLSR", "DSDV", "HYBRID"} THEN
        IF GetActiveL3NextHop(B.destination, OUT l3NextHop) THEN
            TransmitBundleTo(B, B.destination, ticketsToGive = 1)
            forwardedPrimary ← TRUE
        END IF

    // Step 3: Geographic Stateless Forwarding (GRP / GPSR) [28]
    ELSE IF m_mode IN {"GRP", "GPSR"} THEN
        bestNbIp ← 0.0.0.0, minDist ← Distance(myPos, dstPos)
        FOR EACH nb IN neighbors DO
            IF Distance(nb.position, dstPos) < minDist THEN
                minDist ← Distance(nb.position, dstPos)
                bestNbIp ← nb.ip
            END IF
        END FOR
        IF bestNbIp ≠ 0.0.0.0 THEN
            TransmitBundleTo(B, bestNbIp, ticketsToGive = 1)
            forwardedPrimary ← TRUE
        ELSE IF hasOtaGps = TRUE THEN
```

```
            B.gpsrMode ← PERIMETER
            perimNbIp <- SelectRightHandRulePerimeterNeighbor(myPos, dstPos, B.prevHop)
            TransmitBundleTo(B, perimNbIp, ticketsToGive = 1)
            forwardedPrimary <- TRUE
        END IF

    // Step 4: Security Watchdog Routing [15]
    ELSE IF m_mode = "SECURITY" THEN
        IF GetActiveL3NextHop(B.destination, OUT gw) AND m_trustScores[gw] ≥ 0.5 THEN
            TransmitBundleTo(B, gw, ticketsToGive = 1)
            forwardedPrimary ← TRUE
        ELSE
            bestTrustedIp ← SelectMinCostTrustedNeighbor(neighbors, dstPos)
            IF bestTrustedIp ≠ 0.0.0.0 THEN
                TransmitBundleTo(B, bestTrustedIp, ticketsToGive = 1)
                forwardedPrimary ← TRUE
            END IF
        END IF

    // Step 5: Heuristic Adaptive Routing
    ELSE IF m_mode = "ADAPTIVE" THEN
        bestAdaptiveIp ← SelectAdaptiveNeighbor(neighbors, dstPos, m_battery)
        IF bestAdaptiveIp ≠ 0.0.0.0 THEN
            TransmitBundleTo(B, bestAdaptiveIp, ticketsToGive = 1)
            forwardedPrimary ← TRUE
        END IF
    END IF

    // Step 6: Binary Spray & Wait DTN Fallback
    IF m_enableDtn = TRUE OR m_mode = "ADAPTIVE" THEN
        ExecuteBinarySprayAndWait(B, neighbors)
    END IF

    IF forwardedPrimary = TRUE AND m_enableDtn = FALSE THEN
        m_buffer.erase(B)
    END IF
END FOR
ScheduleNextCheck(m_checkInterval = 0.25s)
```

Algorithm 2: Binary Spray & Wait DTN Ticket-Splitting & Store-Carry-Forward

In the event that physical partitioning causes failures in either direct or multiple hops connections, this protocol is applied to ensure message survivability in radio shadow areas. [14].

INPUT: Bundle B (with B.tickets = L), Active Neighbor List neighbors
OUTPUT: Ticket-Bounded Replicated Bundles / Persistent Buffer Entry

```
1:   FOR EACH nb IN neighbors DO
2:       // Ignore malicious nodes if security monitoring is active
3:       IF m_mode = "SECURITY" AND m_trustScores[nb.nodeId] < 0.5 THEN
4:           CONTINUE
5:       END IF
6:
7:       // Binary Spray Phase (L > 1)
```

```
8:        IF B.tickets > 1 AND nb.nodeId NOT IN B.sprayedToNodes THEN
9:            sprayedTickets ← ⌊ B.tickets / 2 ⌋
10:           B.tickets ← B.tickets − sprayedTickets
11:           B.sprayedToNodes.insert(nb.nodeId)
12:
13:           // Transmit copy with halved ticket allocation over 1-hop UDP link
14:           TransmitBundleTo(B, nb.ip, ticketsToGive = sprayedTickets)
15:       END IF
16:   END FOR
17:
18:   // Wait Phase (L = 1)
19:   IF B.tickets = 1 THEN
20:       Retain B in local FIFO Queue m_buffer
21:       // Physical node movement carries bundle across coverage gap
22:       // Direct delivery occurs when destination enters 1-hop neighbor table
23:   END IF
```

Algorithm 3: Offline SOS Message Forwarding and Acknowledgement in MeshSOS

This is how the real process of our Android application will be explained: receiving an SOS, duplicate detection, local storage, transmission to other peers, and acknowledgment propagation.

INPUT: Received SOS message M, ingress peer P, local database DB, connected peers N
OUTPUT: Stored SOS message, forwarded message copies, acknowledgement processing

```
1:    Receive SOS message M from peer P.
2:    Validate the message and check its expiry.
3:    Check whether M has already been processed.
4:    IF M is a duplicate:
          Skip duplicate processing.
      ELSE:
          Store M in the local database DB.
          Notify the user of the received SOS.
5:    IF the hop limit allows further forwarding:
          FOR EACH peer Q IN N:
              IF Q is not the ingress peer P:
                  Forward M to Q.
6:    Process the acknowledgement associated with M.
7:    IF an acknowledgement is received:
          Check whether it matches a locally originated SOS.
          Update the local acknowledgement state when applicable.
          Forward the acknowledgement to peers other than its ingress peer.
8:    Retain eligible messages for store-and-forward when peers reconnect.
```

PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

1. System Architecture Block Diagram

Figure 6.1 shows the layered system architecture, showing the interaction between physical transceivers, network routing levels, the unified application overlay and mobile user endpoints.

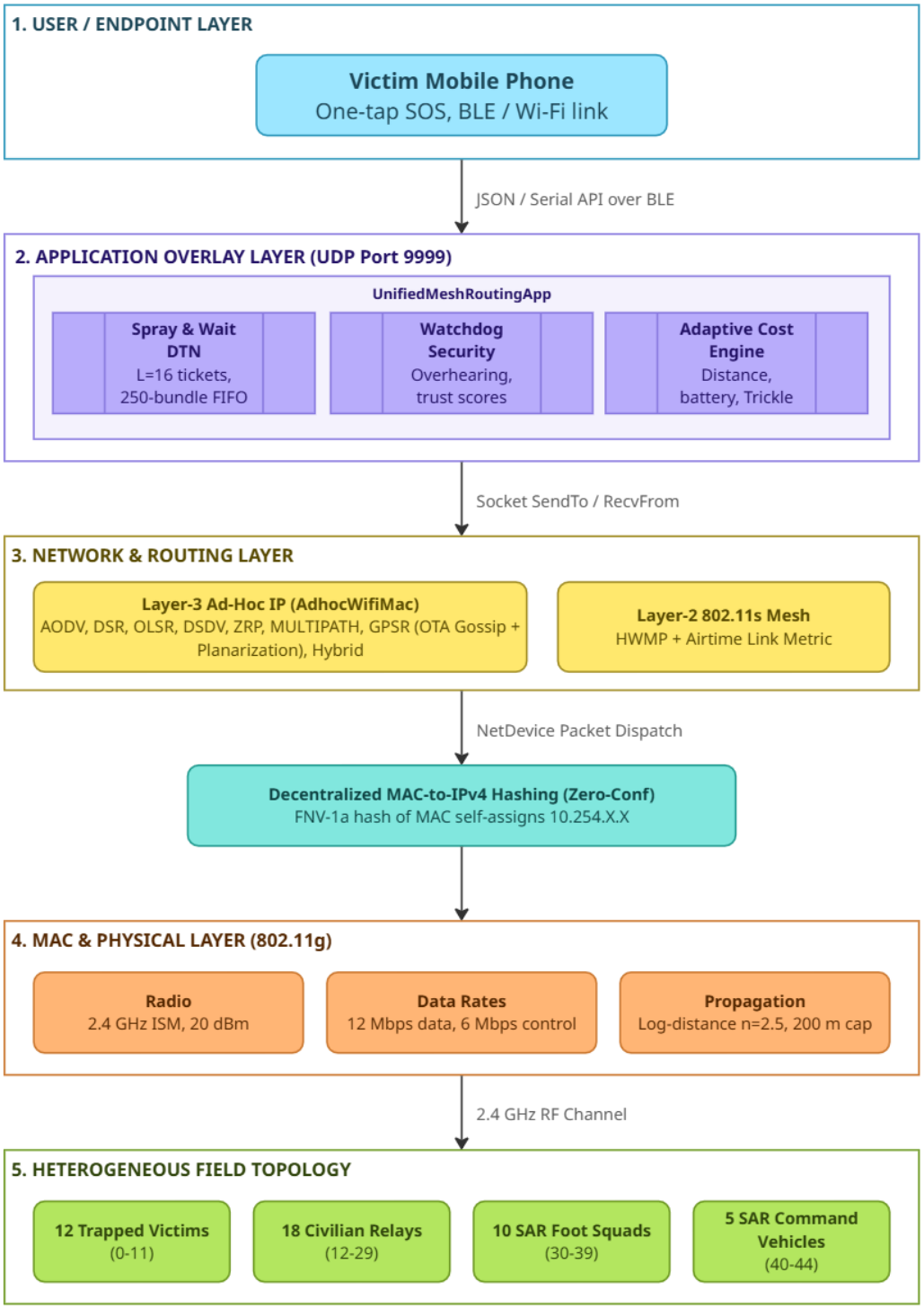


Figure 6.1: System Architecture Block Diagram

2. Context Diagram

Figure 6.2 is the definition of the system boundary, showing the interaction between the external components and the emergency mesh system.

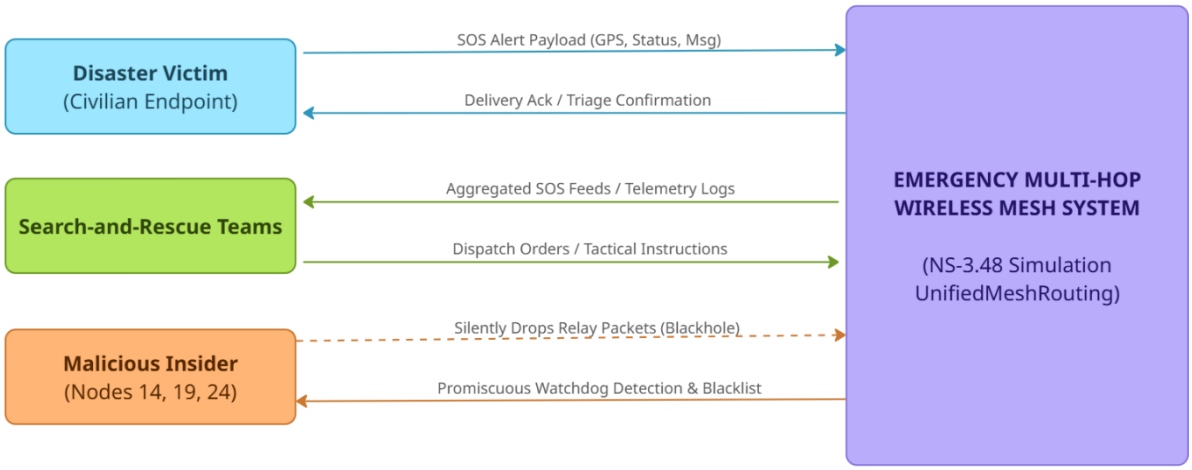


Figure 6.2: Context Diagram of Emergency Wireless Mesh System

3. Subsystem Decomposition

The DFD in Figure 6.3 breaks the system down into four primary processing subsystems as follows: Message Generation & Ingestion, Route Resolution & Mode Selection, Store Carry Forward DTN Queue, and Watchdog Trust Assessment.

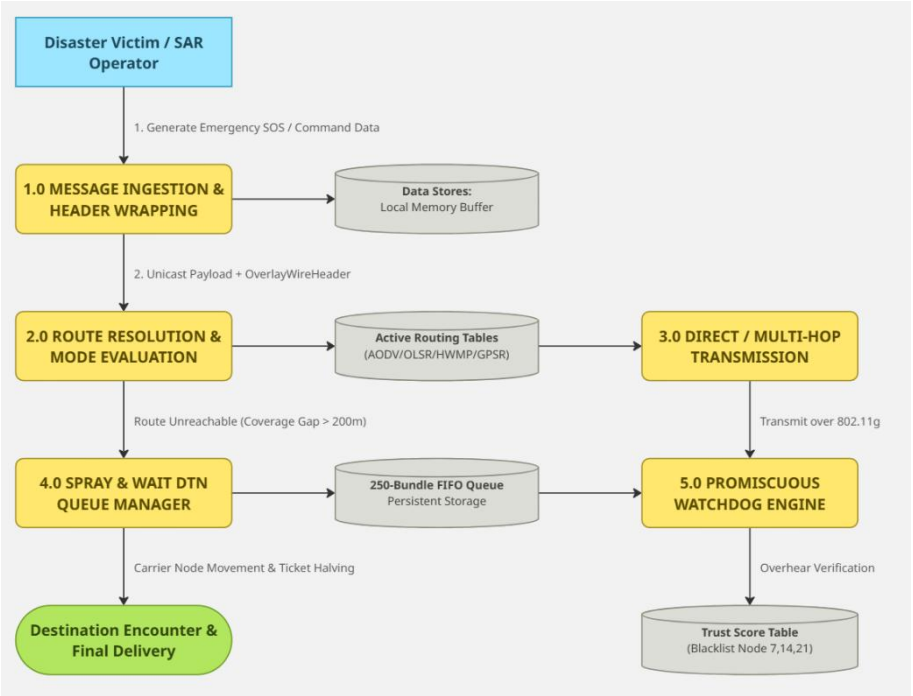


Figure 6.3: Subsystem Decomposition

4. DTN Store-Carry-Forward & Watchdog Operations

The DFD in Figure 6.4 describes the inner workings of the Binary Spray and Wait DTN and Watchdog Security Monitor.

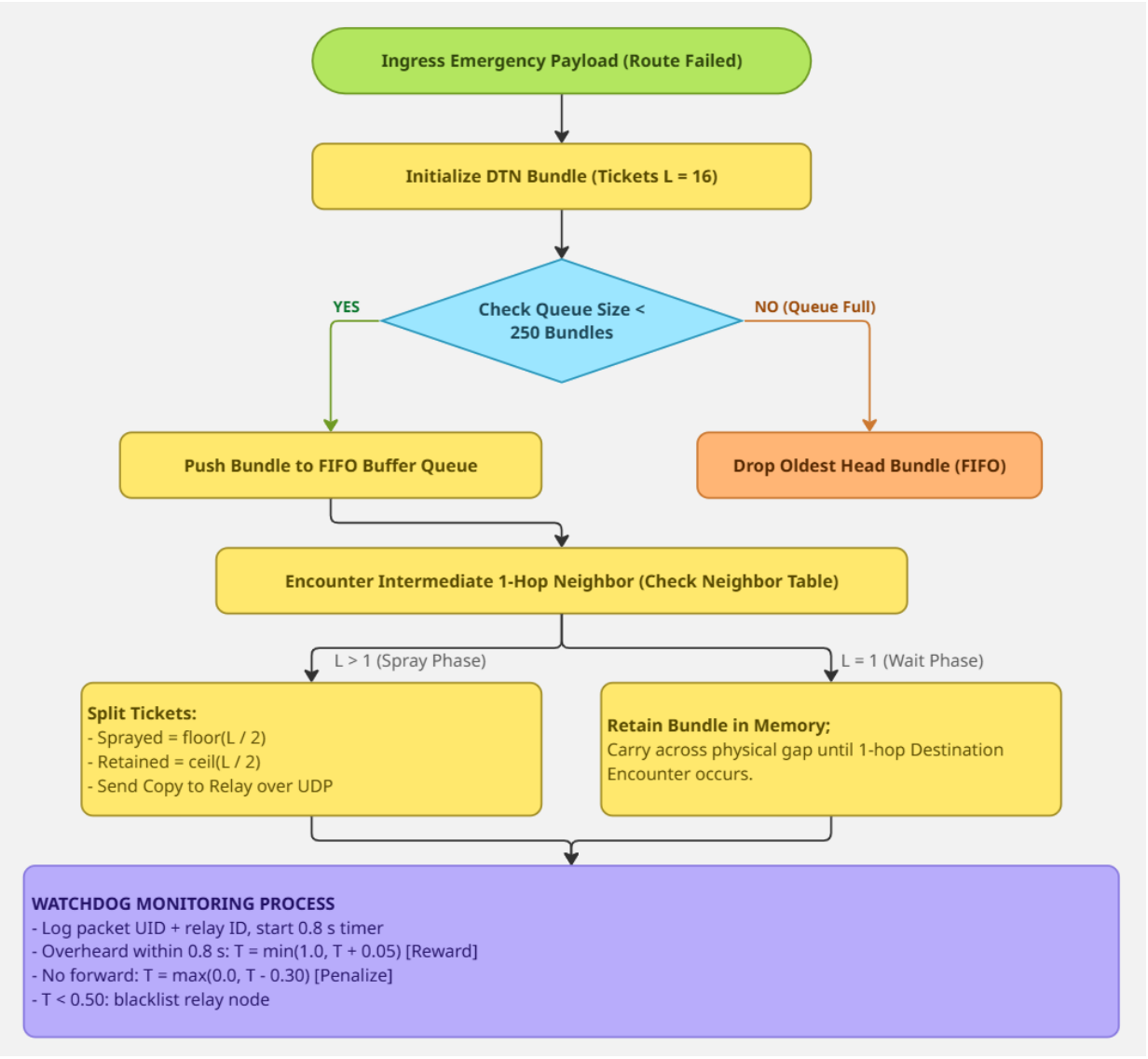


Figure 6.4: DTN and Watchdog Internal Process

5. End-to-End Application Packet Flowchart

This flowchart maps the precise decision tree executed when an emergency packet is generated or received.

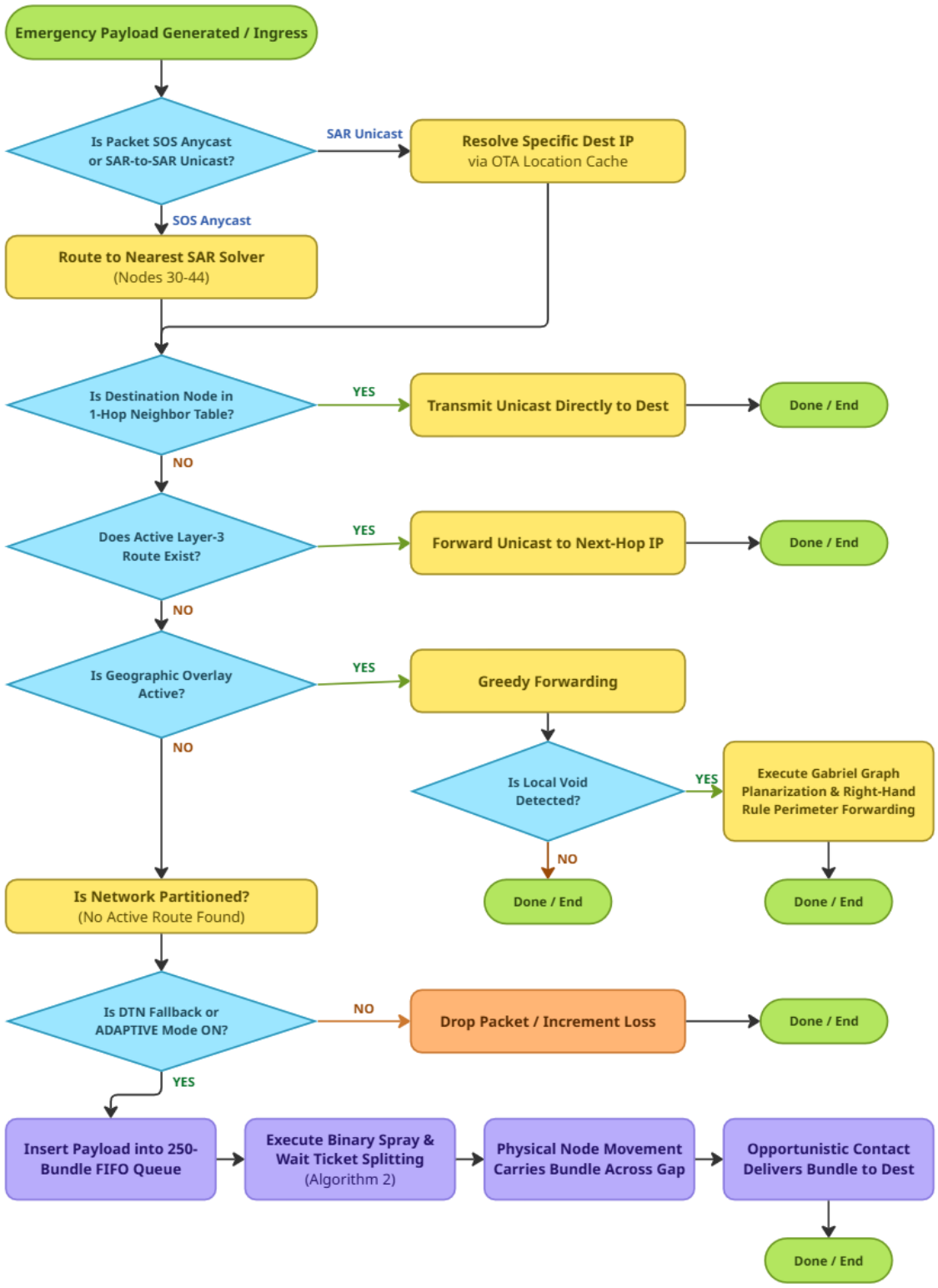


Figure 6.5: End-to-End Application Packet Processing Flowchart

6. MeshSOS Android Application Workflow

The flowchart below explains the functioning of the MeshSOS android application created for offline emergency communications. It represents the process of SOS message generation, device discovery, connection establishment using Wi-Fi Aware or Bluetooth Classic, message transmission, duplicate detection, local storage, acknowledgement, and multi-hop forwarding. The application also supports storing pending messages and forwarding them when connected peers become available.

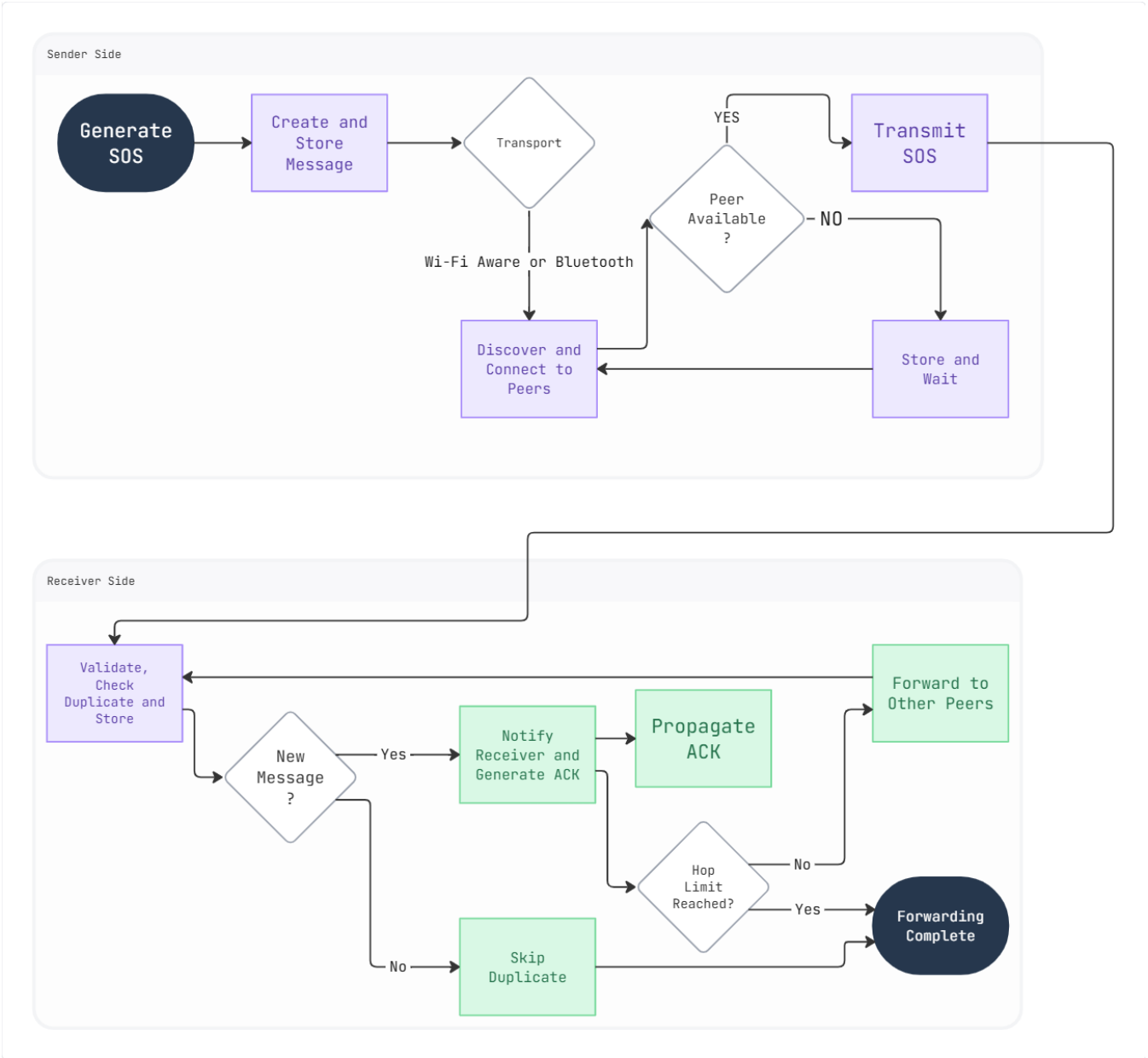


Figure 6.6: MeshSOS Android Application Workflow

IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

1. Software Environment and Simulation Toolchain

The simulation and development environment is built upon a Linux toolchain within Windows Subsystem for Linux (Windows 11).

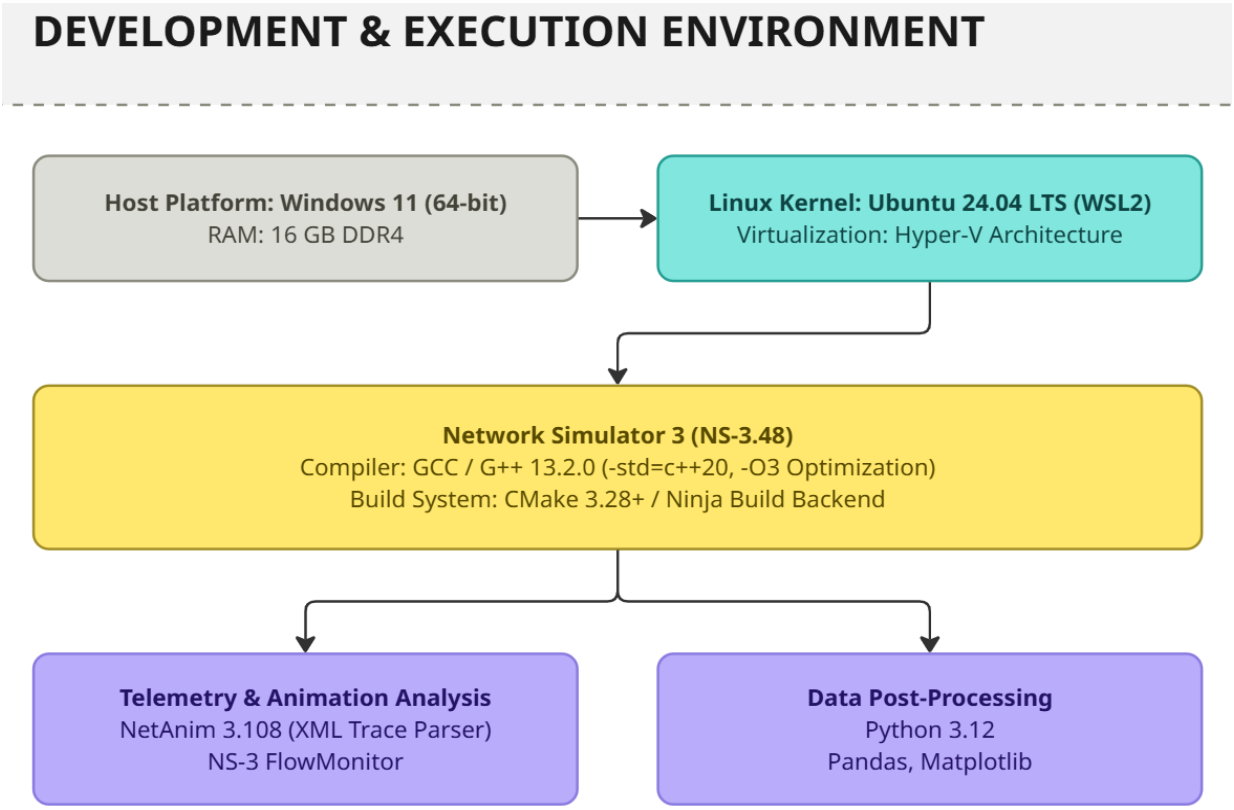


Figure 7.1: Development & Execution Environment

Core Tools and Framework Specifications

- Network Simulator 3 (NS-3.48): Network simulation software used. It features highly accurate C++ implementations of IEEE 802.11 MAC/PHY channels, IPv4 internet stack, and IEEE 802.11s mesh nodes. [2], [1].
- Windows Subsystem for Linux (WSL) & Ubuntu 24.04 LTS: Used as the operating system environment running the NS-3 C++ builds. It eliminates virtual machine emulation overhead by running a native Linux kernel over Hyper-V architecture, providing a full POSIX socket and thread scheduling compatibility.
- GNU Compiler Collection (GCC/G++ 13.2.0): It is the C++ compiler toolchain used. Standard C++ 20 with -O3 execution flags enabled to process high-volume packet trace loops.

- CMake 3.28+ & Ninja Build Manager: Used as the build system generator for NS-3. It compiles C++ source files into multi-threaded modular shared libraries.
- NetAnim 3.108 (Network Animator): Used as the graphical visualizer. It parses the XML trace files generated by ns3::AnimationInterface, renders 2D node movement, transmits/receives packets, etc [1].
- NS-3 FlowMonitor Framework: It is used for packet-level flow analysis. It tracks end-to-end packet transmission, reception, delay sum and loss at the IP layer.
- Python 3.12 Data Analysis Stack: It is used for post-simulation data parsing and plot generation.

2. Embedded Hardware Reference Specifications

To ensure that simulation parameters accurately model real-world transceivers, physical-layer constants are calibrated according to real-world off-grid hardware components (such as Raspberry Pi Single Board Computers, Espressif ESP32 microcontrollers and SX1262 transceivers [29]).

Table 7.1: Hardware Specifications & Simulation Parameter Mapping

Hardware Component	Reference Product Specification	Simulated Physical Parameter Mapping
Primary Microcontroller / SoC	Espressif ESP32-WROOM-32D / ESP32-S3	Embedded memory and socket queue limits (250 bundles for RPI profile and 50 bundles for ESP32 profile)
Wi-Fi Radio System	2.4 GHz 802.11b/g/n Transceiver	WifiHelper::SetStandard(WIFI_STANDARD_80211g)
RF Power Amplifier	CMOS On-Chip PA (+20 dBm Output)	wifiPhy.Set("TxPowerStart", DoubleValue (20.0))
Antenna Subsystem	3 dBi Omnidirectional Antenna	wifiPhy.Set("TxGain", DoubleValue(3.0))
Carrier Vehicle SBC	Raspberry Pi 4 / Pi 5 (4 GB RAM)	Fast moving nodes (Nodes 40-44)

3. Physical Layer and Wireless Channel Configuration in C++

The physical wireless channel in C++ is made using NS-3's YansWifiChannelHelper and YansWifiPhyHelper classes to model Non-Line-of-Sight (NLoS) signal attenuation in damaged urban environments.

```
// 1. Configure Channel Propagation Delay and Loss Models
YansWifiChannelHelper wifiChannel;
wifiChannel.SetPropagationDelay("ns3::ConstantSpeedPropagationDelayModel");

// Log-Distance Loss: Reference Loss = 40 dB at 1m, Path Loss Exponent n = 2.5
wifiChannel.AddPropagationLoss(“ns3::LogDistancePropagationLossModel”,
                               “Exponent”, DoubleValue(2.5),
                               “ReferenceDistance”, DoubleValue(1.0),
                               “ReferenceLoss”, DoubleValue(40.0));

// Hard Physical Range Limit: 200.0 meters maximum radio reach
wifiChannel.AddPropagationLoss(“ns3::RangePropagationLossModel”,
                               “MaxRange”, DoubleValue(200.0));

// Configure Physical Transceiver Properties
YansWifiHelper wifiPhy;
wifiPhy.SetChannel(wifiChannel.Create());
wifiPhy.Set(“TxPowerStart”, DoubleValue(20.0));
wifiPhy.Set(“TxPowerEnd”,   DoubleValue(20.0));
wifiPhy.Set(“TxGain”,       DoubleValue(3.0));
wifiPhy.Set(“RxGain”,       DoubleValue(3.0));

// Configure IEEE 802.11g MAC Modulation Rates
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211g);
wifi.SetRemoteStationManager(“ns3::ConstantRateWifiManager”,
                             “DataMode”, StringValue(“ErpOfdmRate12Mbps”),
                             "ControlMode", StringValue("ErpOfdmRate6Mbps"));
```

4. Detailed C++ Software Implementation

The primary simulation software consists of a C++ file with over 1000 lines of code. It integrates custom application overlays, socket communication interfaces, protocol helpers, etc.

Application Overlay Class

The custom overlay is implemented as a subclass of ns3::Application, managing socket callbacks, in-node queues, hello beacons, watchdog checks, Over-the-Air (OTA) location cache for geographic routing and Spray & Wait DTN routing.

```
class UnifiedMeshRoutingApp : public Application
{
public:
    UnifiedMeshRoutingApp();
```

```
    void Setup(Ptr<Socket> socket, uint16_t L, NodeContainer allNodes,
               uint32_t nodeIndex, std::string mode, bool enableDtnFlag);

    void SendEmergencyAlert(uint16_t flowId, const Ipv4Address destIp,
                            const uint16_t destPort, uint32_t packetSize);

protected:
    void StartApplication() override;
    void StopApplication() override;

private:
    void SendHelloBeacon();
    void ProcessRoutingQueue();
    void HandleReceive(Ptr<Socket> socket);
    void TransmitBundleTo(const DtnBundle& b, Ipv4Address nextHopIp, uint16_t ticketsToGive);
    void VerifyWatchdogForwarding(uint32_t seqNum, uint32_t relayIdx);

    Ptr<Socket> m_socket;
    uint16_t m_initialTickets;
    uint32_t m_maxBufferSize;
    Time m_checkInterval;
    Time m_beaconInterval;
    EventId m_timerEvent;
    EventId m_beaconEvent;
    NodeContainer m_allNodes;
    uint32_t m_nodeId;
    std::string m_mode;
    bool m_enableDtn;
    double m_battery;
    std::deque<DtnBundle> m_buffer;
    std::set<uint32_t> m_seenPackets;
    std::map<uint32_t, NeighborEntry> m_neighborTable;
    std::map<uint32_t, double> m_trustScores;
    std::map<uint64_t, Time> m_pendingWatchdog;
};
```

5. Android Application Implementation (MeshSOS)

5.1 Development Environment and Technologies

The Android application, MeshSOS, was developed to provide offline communication between nearby Android devices during emergency situations. The application is implemented using Kotlin and Android's native development framework. Android Studio is used as the primary development environment.

The application uses Wi-Fi Aware for nearby device discovery and direct communication, while Bluetooth Classic is provided as an alternative communication transport. A local Room database is used to maintain message records on the device. The application follows a modular structure in which transport handling, message routing, and data persistence are managed separately.

The main development technologies and their roles are listed in Table 7.2

Table 7.2: Development Technologies Used in MeshSOS

Technology / Component	Purpose
Kotlin	Primary programming language for Android development
Android Studio	Application development and debugging environment
Android SDK	Access to Android platform APIs
Wi-Fi Aware	Nearby device discovery and direct communication
Bluetooth Classic	Alternative device-to-device communication
TCP Sockets	Data exchange over an established Wi-Fi Aware connection
Room Database	Local storage of SOS messages and delivery records
Android Foreground Service	Maintains connection-related operations during background execution

5.2 Device Discovery and Communication

MeshSOS supports two communication transports: Wi-Fi Aware and Bluetooth Classic. The Wi-Fi Aware implementation uses Android's Wi-Fi Aware APIs to discover nearby devices advertising or subscribing to the application service. The service name used for discovery is Offline_SOS_Mesh.

Following the discovery of a matching device, a connection request and response exchange occurs. Following the successful acceptance of the connection, the application uses the Android network callback and network specifier APIs to create a direct connection through which data can be transmitted. Next, the application creates a TCP socket on this connection for the purpose of transmitting SOS messages.

The Bluetooth Classic protocol is used as an alternative transport that enables the use of Bluetooth discovery to connect devices using the RFCOMM socket API to enable message transfer between the two devices without requiring any internet connection.

For reliable data exchange, the application implements length prefix messaging format. Every message sent starts with a 4-byte length prefix which helps the receiving party to detect the boundaries of the message sent.

Code Snippet 1: Wi-Fi Aware Service Discovery

Source: *WiFiAwareTransport.kt*

The following code demonstrates the publication and subscription configuration used by MeshSOS for Wi-Fi Aware service discovery. Each device advertises its application identity and searches for nearby devices using the common service name `Offline_SOS_Mesh`.

```
val identity = WifiAwareIdentity(
    version = WifiAwareIdentity.CURRENT_VERSION,
    deviceId = preferences.deviceId,
    deviceName = preferences.deviceName
)

val pubConfig = PublishConfig.Builder()
    .setServiceName(SERVICE_NAME)
    .setServiceSpecificInfo(identity.toByteArray())
    .build()

val subConfig = SubscribeConfig.Builder()
    .setServiceName(SERVICE_NAME)
    .build()

currentAwareSession.publish(
    pubConfig,
    object : DiscoverySessionCallback() {
        override fun onPublishStarted(
            session: PublishDiscoverySession
        ) {
            publishSession = session
        }
    },
    mainHandler
)

currentAwareSession.subscribe(
    subConfig,
    object : DiscoverySessionCallback() {
        override fun onSubscribeStarted(
            session: SubscribeDiscoverySession
        ) {
            subscribeSession = session
        }
    },
    mainHandler
)
```

When a matching service is discovered, the application extracts the peer identity and adds the device to its discovered-peer list. This information is later used during connection establishment.

Code Snippet 2: Wi-Fi Aware TCP Connection Establishment

Source: *WiFiAwareTransport.kt*

After the discovery handshake is completed, the responding device creates a server socket and requests a Wi-Fi Aware network connection using a network specifier. The negotiated socket connection is then used for direct communication between the devices.

```
val serverSocket = ServerSocket().apply {
    reuseAddress = true
    bind(InetSocketAddress(0))
}

val serverPort = serverSocket.localPort

val specifier = WifiAwareNetworkSpecifier.Builder(
    session, peerHandle
)
    .setPskPassphrase(PASSPHRASE)
    .setPort(serverPort)
    .build()

val networkRequest = NetworkRequest.Builder()
    .addTransportType(
        NetworkCapabilities.TRANSPORT_WIFI_AWARE
    )
    .setNetworkSpecifier(specifier)
    .build()

connectivityManager?.requestNetwork(
    networkRequest, callback, 25000
)
```

The application keeps separate connection profiles for individual peers, enabling more than one Wi-Fi Aware connection to run simultaneously, depending on the data path capability of the device.

Code Snippet 3: Length-Prefixed Message Framing

Source: *MessageFraming.kt*

Since TCP and Bluetooth RFCOMM operate as byte streams, message boundaries need to be maintained separately. MeshSOS implements a length-prefixed framing mechanism in which each payload is preceded by a 4-byte big-endian integer representing its length. The maximum permitted frame size is 16 KiB.

```
const val MAX_FRAME_SIZE = 16384
const val HEADER_SIZE = 4
```

```
 const val MAX_FRAME_SIZE = 16384
const val HEADER_SIZE = 4

fun encodeFrame(payload: ByteArray): ByteArray {
    require(payload.size <= MAX_FRAME_SIZE)

    val buffer = ByteBuffer.allocate(
        HEADER_SIZE + payload.size
    )

    buffer.order(ByteOrder.BIG_ENDIAN)
    buffer.putInt(payload.size)
    buffer.put(payload)

    return buffer.array()
}

fun writeFrame(
    outputStream: OutputStream,
    payload: ByteArray
) {
    val framed = encodeFrame(payload)
    outputStream.write(framed)
    outputStream.flush()
}
fun encodeFrame(payload: ByteArray): ByteArray {
    require(payload.size <= MAX_FRAME_SIZE)

    val buffer = ByteBuffer.allocate(
        HEADER_SIZE + payload.size
    )

    buffer.order(ByteOrder.BIG_ENDIAN)
    buffer.putInt(payload.size)
    buffer.put(payload)

    return buffer.array()
}

fun writeFrame(
    outputStream: OutputStream,
    payload: ByteArray
) {
    val framed = encodeFrame(payload)
    outputStream.write(framed)
    outputStream.flush()
}
```

On the receiving side, the length header is read first, followed by the associated payload data. This ensures that messages are accurately reassembled despite the arrival of data in chunks.

5.3 SOS Routing and Message Reliability

The application implements an application-level message forwarding system to ensure that SOS messages can propagate beyond the initial peers that the message is received at. Once an SOS message is received, the application will validate it and check if the message has been already processed before. In case if duplicate messages are received, they are being discarded, and only new messages will be processed by storing them into the local database and showing the notification to the receiver.

MeshRouter is responsible for the message forwarding between connected peers. In case if a message has been forwarded, the application does not forward the same message back to the peer it came from in order to avoid additional retransmissions. The hop limit and expiry time of the message are used to control the message spreading.

Also, the application implements an acknowledgment scheme. Once a recipient confirms an SOS alert, the application will generate an ACK message associated with the SOS message and send this acknowledgment further through connected peers. Once the original device gets an ACK message, it changes the message delivery status to acknowledge. Also, duplicate acknowledgments are ignored.

In order to handle temporary network disconnection, MeshSOS keeps message records in the local storage and implements a retry mechanism for pending messages. These messages can be sent later if the connection between peers becomes possible again.

Code Snippet 4: Duplicate Detection and Local Message Storage

Source: *MeshRouter.kt*

When an SOS message is received, MeshRouter checks its message ID against both an in-memory set and the Room database. A mutex is used to protect the duplicate-checking operation. If the message is new, it is converted into a database entity and stored locally before further processing.

```
val isNew = dedupMutex.withLock {
    if (seenMessageIds.contains(messageId)) {
        return@withLock false
    }

    if (dao.exists(messageId) > 0) {
        seenMessageIds.add(messageId)
        return@withLock false
    }
```

```
    seenMessageIds.add(messageId)
    true
}

if (!isNew) {
    return
}

val entity = SosMessageEntity.fromSosMessage(
    message = envelope.toSosMessage(),
    direction = MessageDirection.INCOMING,
    status = DeliveryStatus.SENT,
    senderName = resolvedName,
    isAcknowledged = false,
    originDeviceId = envelope.originDeviceId,
    hopLimit = envelope.hopLimit
)

dao.insertOrUpdate(entity)
```

This mechanism prevents repeated processing of the same SOS message, including messages received again after application restarts.

Code Snippet 5: Multi-Hop SOS Message Forwarding

Source: *MeshRouter.kt*

After successfully storing a new SOS message, MeshRouter checks whether the message can be forwarded further. If forwarding is allowed, the hop count is updated and the message is broadcast to connected peers, excluding the peer from which it was received.

```
if (envelope.canForward()) {
    val relayEnvelope = envelope.nextHop()
    val relayBytes = relayEnvelope.toByteArray()

    val results = transportManager.broadcastFrame(
        relayBytes,
        excludedPeerId = ingressPeerId
    )

    for ((peerId, success) in results) {
        if (success) {
            dao.recordPeerDelivery(
```

```
                MeshPeerDeliveryEntity(
                    messageId = messageId,
                    peerId = peerId
                )
            )
        }
    }
}
```

Successful transfers are logged in the database with their corresponding peer ID numbers. Such information is utilized by the store-and-forward process for the purpose of avoiding unnecessary retransmission to the same peer again.

Code Snippet 6: Acknowledgement Processing

Source: *MeshRouter.kt*

MeshSOS sends out Acknowledgement messages to inform of the successful receipt of the SOS messages. Upon receipt of the ACK message, the application verifies whether it is an already processed message. In case it relates to the SOS messages that are generated locally, its delivery is marked accordingly. The ACK is propagated to other peer nodes connected, within the hop limit.

```
val targetMessageId =
    envelope.ackForMessageId ?: envelope.messageId

val ackKey = "$targetMessageId:${envelope.originDeviceId}"

val isNewAck = seenAckKeys.add(ackKey)

if (!isNewAck) {
    return
}

val localMessage = dao.getMessageById(targetMessageId)

if (localMessage != null &&
    localMessage.direction == MessageDirection.OUTGOING.name
) {
    dao.markAcknowledged(
        targetMessageId,
        acknowledgedAt = envelope.createdAt,
```

```
        status = DeliveryStatus.ACKNOWLEDGED.name
    )
}

if (envelope.canForward()) {
    val relayAck = envelope.nextHop()

    transportManager.broadcastFrame(
        relayAck.toByteArray(),
        excludedPeerId = ingressPeerId
    )
}
```

This method ensures that the acknowledgement information is spread over different devices and reaches the sending device without the need for a direct link between the two devices.

Code Snippet 7: Retry Mechanism Using Exponential Backoff

Source: *SosRepository.kt*

For addressing communication issues that may occur temporarily, the application performs periodic queries to check whether there are any SOS messages that need to be sent. This is done using the bounded exponential backoff technique, which increases the wait time exponentially till a maximum wait time of 30 seconds is reached.

```
private val MAX_RETRIES = 5

val backoffMs = min(
    30_000L,
    2000L * (1 shl item.retryCount)
)

if (now - item.lastRetryTimestamp >= backoffMs) {
    val newRetryCount = item.retryCount + 1

    dao.updateRetryInfo(
        messageId = item.messageId,
        status = DeliveryStatus.SENT.name,
        retryCount = newRetryCount,
        lastRetryTimestamp = now
    )

    val envelope = MeshEnvelope.fromSosMessage(
        sosMessage = item.toSosMessage(),
```

```
        originDeviceId = item.originDeviceId ?: item.senderId,
        originDeviceName = item.senderName,
        hopLimit = item.hopLimit
    )

    transportManager.broadcastFrame(envelope.toByteArray())
}
```

In case the maximum number of retries has been achieved, then the message is considered failed. If not, then the message is retransmitted when the retry period has passed and peers are connected.

RESULT DISCUSSION AND ANALYSIS

This section presents the empirical performance overview of the multi-protocol wireless mesh framework consisting of 9 different ad-hoc protocols (AODV, OLSR, DSDV, DSR, HYBRID, GRP, IEEE 802.11s HWMP, GRP, MULTIPATH) evaluated under two hardware profiles (Raspberry Pi 4/5 vs. ESP32 Microcontroller).

Overall Hardware-Constrained Packet Delivery Ratio (PDR%) Analysis

To evaluate the impact of physical transceiver limits and store-carry-forward architecture in a 500 m x 500 m disaster grid, the system was benchmarked across two hardware profiles:

1. Raspberry Pi (RPI) Profile: +20.0 dBm transmit power, 3 dBi antenna gain, 200.0 m physical radio range, and L=16 DTN routing tickets, 250-bundle local storage queue [27].
2. ESP32 Microcontroller Profile: +16.0 dBm transmit power, 0.0 dBi PCB trace antenna gain, 60.0 m physical radio range, and L=6 DTN routing tickets, 50-bundle local storage queue [18].

Table 8.1: Packet Delivery Ratio Comparison Across Protocols & Hardware

Hardware	Protocol	Baseline PDR (%)	+DTN PDR (%)	+SEC PDR (%)	+ADAPT PDR (%)	Full Stack (+SEC+ADAPT+DTN) PDR (%)
RPI	AODV	91.62	98.88	94.12	97.38	91.5
RPI	OLSR	92.25	99.25	99	99.38	99.5
RPI	DSDV	91.12	98.62	98.25	98.12	98.5
RPI	DSR	97.88	96.75	92.88	96.75	96.75
RPI	HWMP	73.62	85.25	85.25	87.38	87.38
RPI	HYBRID	88	99.5	95.5	97	99.12
RPI	ZRP	96	98.88	98.5	99.25	93.5
RPI	GRP	95.62	99.38	94.62	98.88	98.62
RPI	MULTIPATH	94	99.12	96.5	99.25	96.5
ESP32	AODV	11.62	60.5	47.12	63.38	53.88
ESP32	OLSR	5.5	49.25	43.25	51.5	49.62
ESP32	DSDV	9.88	65.25	44.75	61	59.38
ESP32	DSR	11.5	52.75	40.88	53	47.25
ESP32	HWMP	7.62	55.12	43.88	55	47.38
ESP32	HYBRID	13.75	67.75	58.38	62.5	62.75
ESP32	ZRP	44.5	58.38	43.38	54.62	52.62
ESP32	GRP	47.12	62.25	47.12	64.12	54.38
ESP32	MULTIPATH	47.88	61	46.5	53.62	56.62

Table 8.2: Network Performance Gains by Hardware Profile

Hardware	Protocol	Net Full-Stack Gain (+%)	Baseline Delay (ms)	+DTN Delay (ms)	Baseline Tput (Kbps)	Full Stack Tput (Kbps)
RPI	AODV	-0.12	24.58	2507.23	52.71	57.07
RPI	OLSR	7.25	15.13	25.91	53.21	57.78
RPI	DSDV	7.38	36.22	22.15	52.22	57.12
RPI	DSR	-1.13	14.8	1814.83	56.75	56.83
RPI	HWMP	13.76	72.89	30.85	42.15	51.06
RPI	HYBRID	11.12	28.72	18.62	51.31	57.69
RPI	ZRP	-2.5	13.62	76.67	56.5	53.76
RPI	GRP	3	34.84	26.37	56.38	57.28
RPI	MULTIPATH	2.5	15.07	256.96	55.63	56.62
ESP32	AODV	42.26	1068.83	5687.8	5.01	24.38
ESP32	OLSR	44.12	2.96	3883.74	3.5	28.82
ESP32	DSDV	49.5	3252.66	9177.18	4.6	31.98
ESP32	DSR	35.75	10131.1	4007	7.79	26.28
ESP32	HWMP	39.76	42.43	11754.3	7.88	29.83
ESP32	HYBRID	49	5.45	4277.45	8.49	37.77
ESP32	ZRP	8.12	55	4436.24	27.34	28.79
ESP32	GRP	7.26	44.88	7237.59	29.4	24.13
ESP32	MULTIPATH	8.74	48.55	5766.05	29.63	30.72

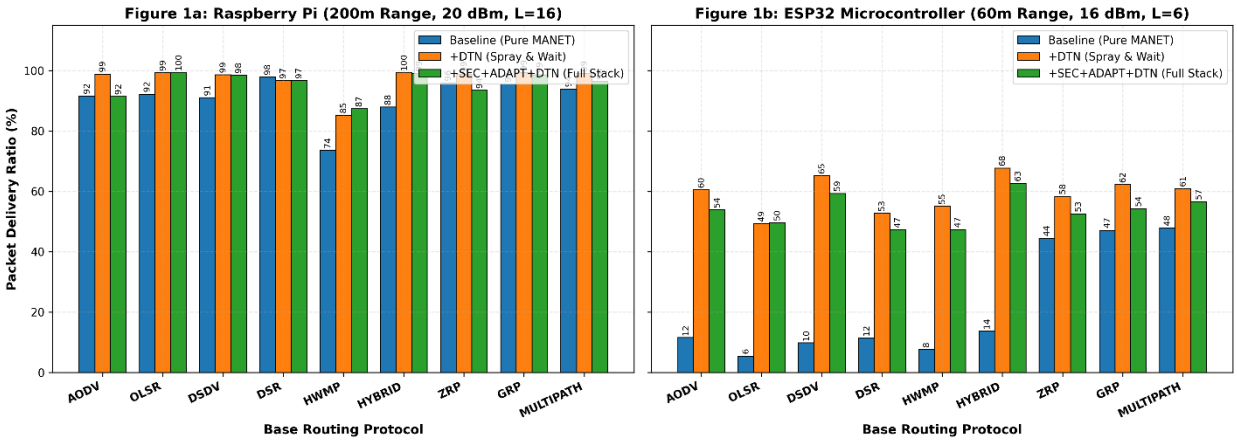


Figure 8.1: Impact of Hardware Constraints (RPI vs. ESP32) and DTN/QoS Overlays on Packet Delivery Ratio

Detailed Analysis

Raspberry Pi Performance (200 m Range): On high-power SBC hardware such as Raspberry Pi with a 200 m radio range [27], baseline MANET protocols achieve a high packet delivery rate (73.6% to 97.9% PDR), as shown in Figure 8.1 and Table 8.1 because a node has an

average of approx. 5-6 other nodes within its radio range. Adding the +DTN store-carry-forward fallback or the Full Stack (+SEC+ADAPT+DTN) increases PDR to very high levels (87.4% to 99.5% PDR), with OLSR having +DTN = 99.25%, Full Stack = 99.50% and HYBRID having +DTN = 99.5%, Full Stack = 99.12% (Figure 8.1) demonstrate almost complete path recovery.

ESP32 Microcontroller Performance (60 m Range): On constrained ESP32 microcontroller with a 60 m radio range, MANET protocols fail catastrophically (baseline) [30] [31], with only 5.5% to 13.8% PDR for traditional protocols (AODV=11.62%, OLSR=5.50%, DSDV=9.88%, DSR=11.50%, HWMP=7.62%, HYBRID=13.75%). Physical distance gaps over 60 m break multi-hop paths, causing standard protocols to drop over 86% of packets.

Net Full-Stack Gain: Activating the +DTN overlay or the Full Stack delivery overlay improves performance drastically on ESP32 nodes, resulting in net gains of +49.50% for DSDV, +49% for HYBRID, +44.12% for OLSR and +42.26% for AODV (see Table 8.2).

Orthogonal Cross-Layer Feature Ablation Analysis

To isolate the individual contribution of each subsystem – Binary Spray & Wait DTN (+DTN), Watchdog Security (+SEC), and Adaptive Battery (+ADAPT), every protocol was tested across all 8 different combinations.

Table 8.3: RPI 8-Mode Orthogonal Ablation Matrix (PDR %)

Base Protocol	Baseline	+DTN	+ADAPT	+ADAPT +DTN	+SEC	+SEC +DTN	+SEC +ADAPT	+SEC +ADAPT +DTN
AODV	91.62	98.88	97.38	96	94.12	99.5	98.88	91.5
OLSR	92.25	99.25	99.38	99.5	99	99.12	99.38	99.5
DSDV	91.12	98.62	98.12	98.75	98.25	98	98.25	98.5
DSR	97.88	96.75	96.75	96.75	92.88	96.75	96.75	96.75
HWMP	73.62	85.25	87.38	87.38	85.25	85.25	87.38	87.38
HYBRID	88	99.5	97	99.38	95.5	99.25	97.38	99.12
ZRP	96	98.88	99.25	99.75	98.5	99.25	99.5	93.5
GRP	95.62	99.38	98.88	98.62	94.62	98.12	99	98.62
MULTIPATH	94	99.12	99.25	99.38	96.5	96	99.12	96.5

Table 8.4: ESP32 8-Mode Orthogonal Ablation Matrix (PDR %)

Base Protocol	Baseline	+DTN	+ADAPT	+ADAPT +DTN	+SEC	+SEC +DTN	+SEC +ADAPT	+SEC +ADAPT +DTN
AODV	11.62	60.5	63.38	58.88	47.12	64.75	54.38	53.88
OLSR	5.5	49.25	51.5	52.5	43.25	47.62	43.62	49.62
DSDV	9.88	65.25	61	53.88	44.75	63.62	44.62	59.38

DSR	11.5	52.75	53	53	40.88	52.12	47.25	47.25
HWMP	7.62	55.12	55	55	43.88	50.38	47.38	47.38
HYBRID	13.75	67.75	62.5	63.5	58.38	61.88	61.12	62.75
ZRP	44.5	58.38	54.62	51.88	43.38	52.12	53.12	52.62
GRP	47.12	62.25	64.12	62.25	47.12	60.5	53.25	54.38
MULTIPATH	47.88	61	53.62	59	46.5	65.38	57.75	56.62

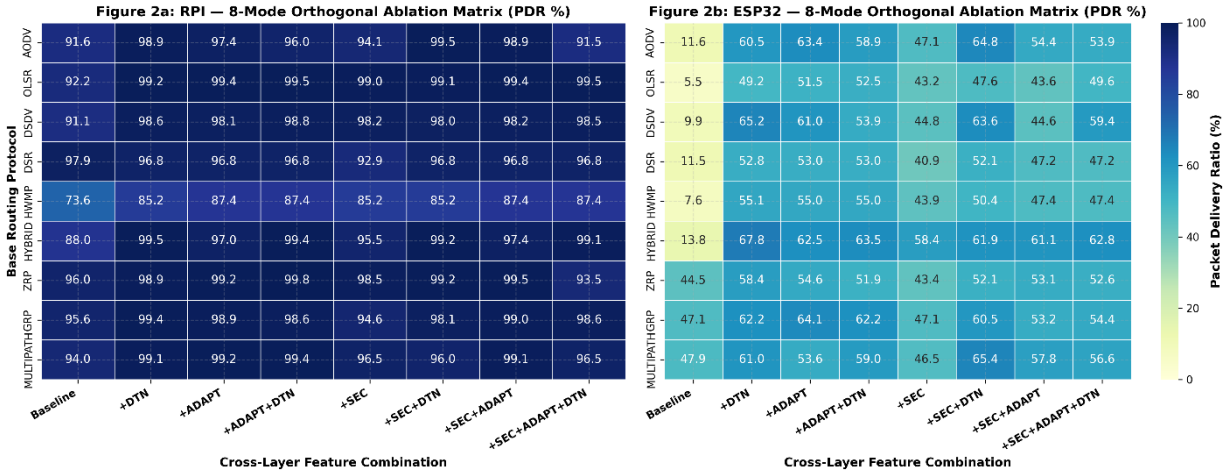


Figure 8.2: Comprehensive Ablation Study Across All 8 Combinations of DTN, Watchdog Security and Adaptive QoS

Detailed Analysis

Raspberry Pi: On Raspberry Pi nodes, activating +DTN or +ADAPT individually increases PDR to 96.0%-99.5%. The combination of +ADAPT +DTN achieves the highest overall stability, with ZRP reaching 99.75% PDR and OLSR reaching 99.50% PDR (see Table 8.3 and Figure 8.2).

ESP32: On ESP32 microcontrollers, +SEC alone improves PDR from approx. 10% to 40.8%-58.4% by filtering out malicious Blackhole nodes. +ADAPT alone increases PDR to 51.5%-64.1% by adjusting beacon intervals and routing around low-battery nodes. +DTN alone achieves the highest single-feature boost (52.75%-67.75% PDR) by bridging network clusters or radio dead zones. Under HYBRID mode on ESP32, combining all three features (+SEC +ADAPT +DTN) maintains a 62.75% PDR, providing security, energy management, and disruption tolerance at the same time (see Table 8.4 and Figure 8.2).

Aggregate Application Throughput (Kbps) Analysis

To evaluate delivery rates across different hardware constraint, aggregate application throughput was extracted across all protocols.

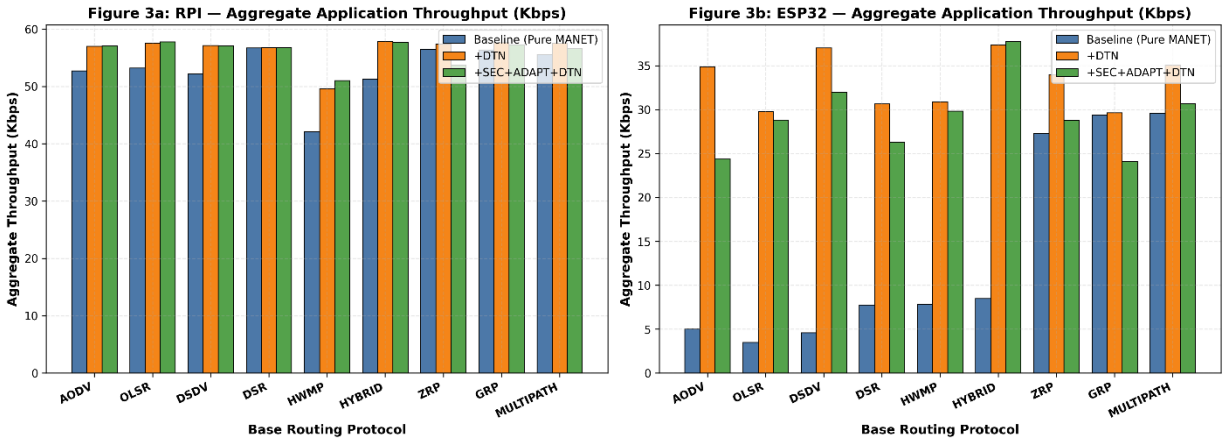


Figure 8.3: Aggregate Application Throughput Comparison (RPI vs. ESP32)

Detailed Analysis

Raspberry Pi Throughput: A radio range of 200 m [27] allows all protocols to maintain a near-maximum throughput (51.06 to 57.78 Kbps against an offered load of approx. 58 Kbps). OLSR, GRP and HYBRID achieve the highest sustained throughput at 57.78 Kbps, 57.69 Kbps, and 57.28 Kbps, respectively (Figure 8.3).

ESP32 Throughput: On ESP32 nodes, baseline protocols suffer extreme throughput throttling (3.50 to 8.49 Kbps for OLSR, DSDV, AODV, HWMP, and HYBRID) because non-routable packets are dropped. Activating the full stack (+SEC +ADAPT +DTN) restores throughput to 37.77 Kbps for HYBRID, 31.98 Kbps for DSDV and 30.72 Kbps for MUTLIPATH, proving that store-carry-and-forward architecture successfully delivers data across broken links (Figure 8.3).

End-to-End Latency in Log Scale (ms) Analysis

Delay metrics highlight the fundamental trade-off between real-time IP forwarding and DTN store-carry-and-forward architecture.

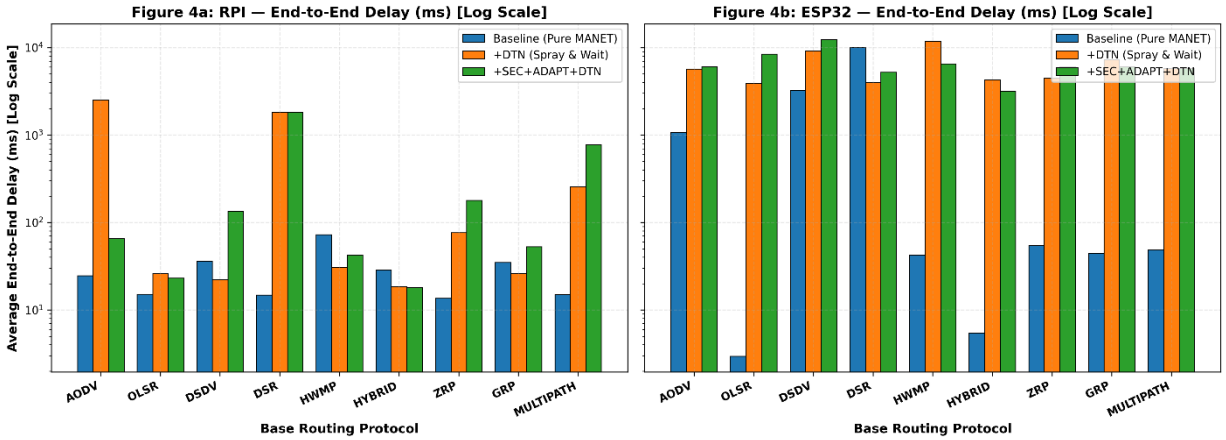


Figure 8.4: Average End-to-End Delay Comparison in Log Scale (ms)

Detailed Analysis

Real-Time Forwarding Latency: When an unbroken end-to-end path exists, standard IP routing delivers packets with low latency (see in Figure 8.4, 13.62 ms to 72.89 ms on RPI for ZRP, DSR, OLSR, MULTIPATH and HWMP).

DTN Store-Carry-and-Forward Latency: When radio coverage gaps break IP paths, the DTN overlay buffers bundles in local queues until carrier nodes can move across dead zones to deliver those bundles. This trades real-time delivery speed for message survivability, increases average latency to 1814 ms–2507 ms on RPI and 3883 ms–11754 ms on ESP32 as shown in Figure 8.4. This delay represents physical movement time across the radio dead zone, ensuring that emergency SOS payloads reach their destination rather than being permanently dropped.

Marginal Feature Gains (Δ% PDR over Baseline)

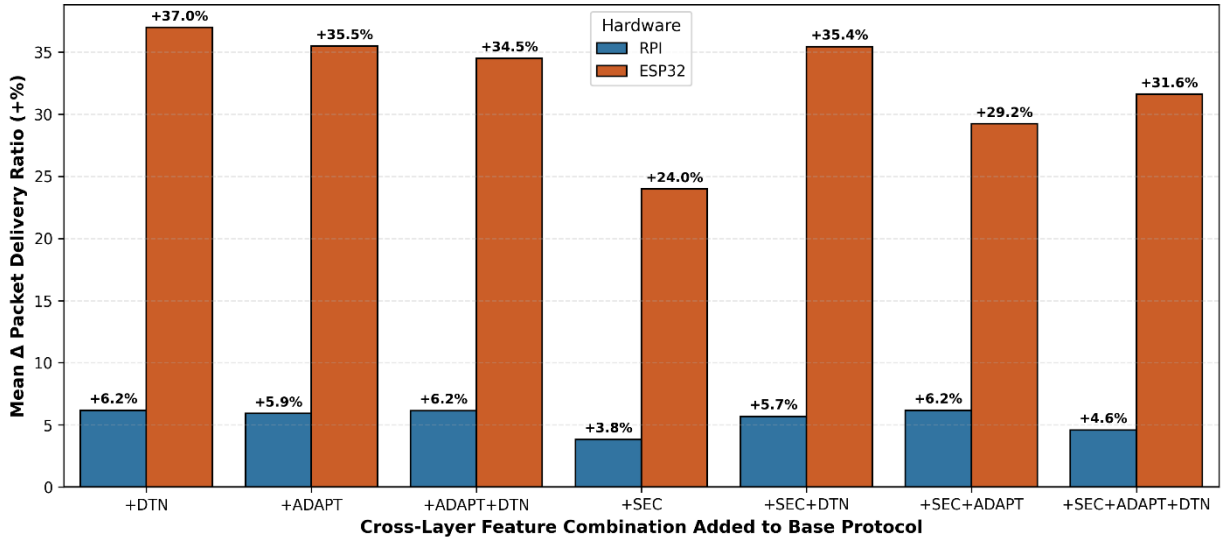


Figure 8.5: Average PDR Improvement Over Pure MANET Baseline Across All 9 Protocols

Detailed Analysis:

Raspberry Pi Marginal Gains: Since RPI baselines already achieve a high PDR (91.1%), adding extra features provides moderate marginal gains (+3.8% to +6.2% PDR) as shown in Figure 8.5.

ESP32 Marginal Gains: On ESP32 hardware, features deliver large marginal gains. +DTN overlay provides a +37.0% mean PDR increase across all 9 protocols. +ADAPT overlay provides a mean +35.5% mean PDR boost. Full Stack (+SEC +ADAPT +DTN) delivers a +31.6% (see Figure 8.5) mean PDR increase while isolating blackhole nodes and managing node energy depletion.

Victim SOS Anycast vs. SAR Team Coordination

To evaluate the operational requirements of a SAR mission in emergency scenario, traffic was divided into two profiles:

- Flows 1-4 (Victim SOS Anycast): Trapped victims broadcast emergency SOS alerts. An SOS payload is satisfied as soon as it reached any SAR node (ROLE_RESCUER_FOOT or ROLE_COMMAND_MULE_VEHICLE).
- Flows 5-6 (SAR Unicast Coordination): Point-to-point tactical coordination between foot team and mobile vehicles.

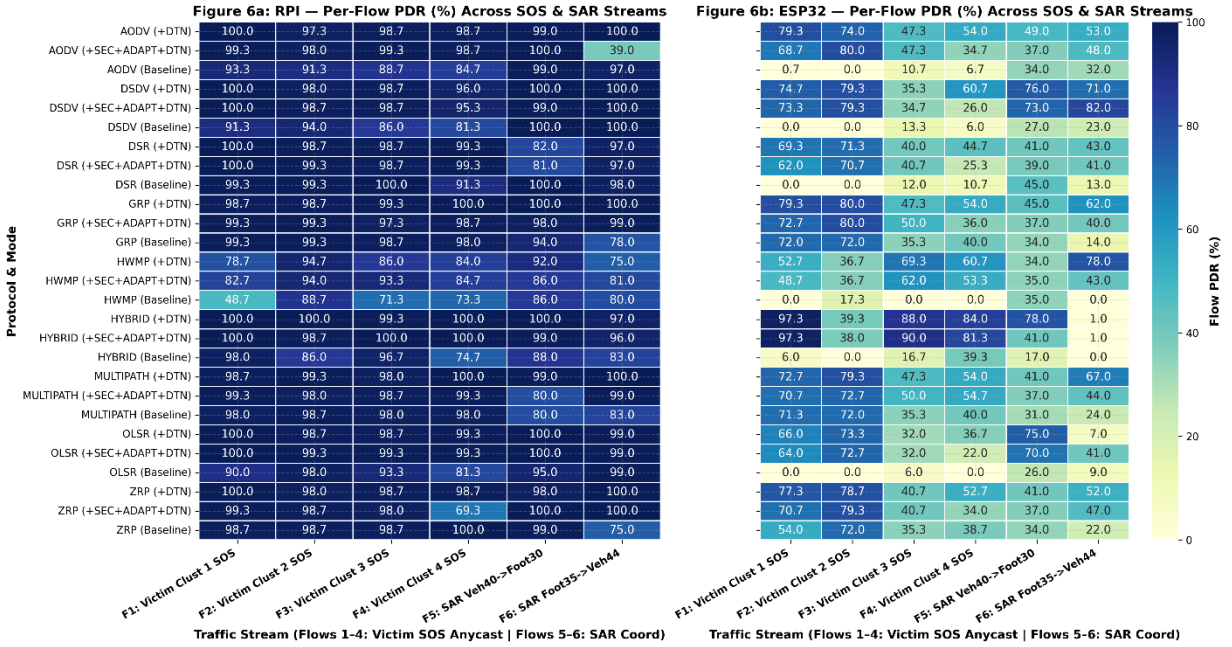


Figure 8.6: Granular Per-Flow PDR Heatmap Across SOS Anycast and SAR Coordination Streams

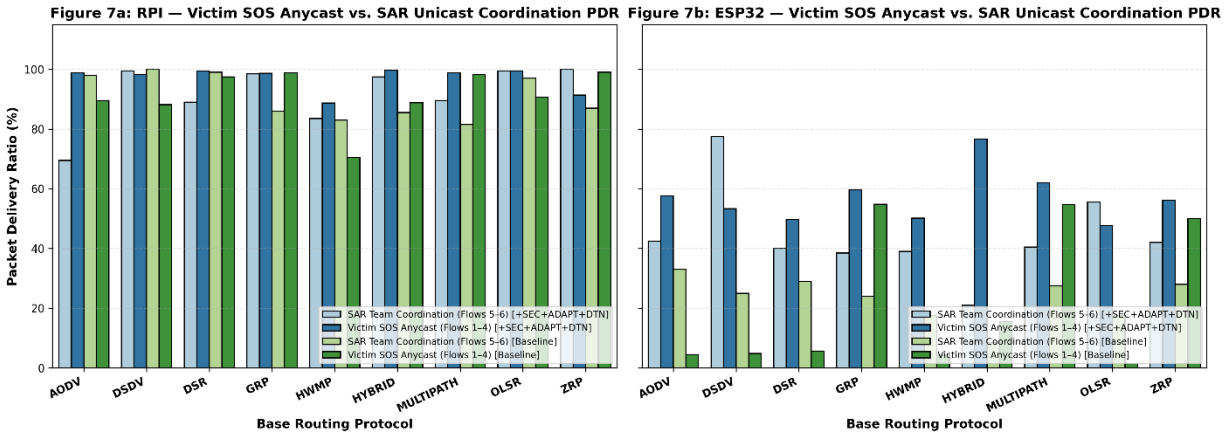


Figure 8.7: Victim SOS Anycast vs. SAR Unicast Coordination PDR Comparison

Detailed Analysis

Victim SOS Anycast Delivery Advantage: Across both hardware profiles, Victim SOS Anycast (Flows 1-4) consistently achieves higher delivery ratios (see Figure 8.6 and Figure 8.7) than SAR Unicast Coordination (Flows 5-6). On ESP32 under HYBRID +DTN, SOS Anycast achieves 97.3% PDR on Flow 1 and 99.3% PDR on Flow 3. Since an SOS distress call can be satisfied by any SAR team which receives the SOS, diversity of options increases delivery probability.

SAR Unicast Coordination: Point-to-point SAR Unicast Coordination requires delivery to a specific node. Under baseline ESP32 conditions, unicast PDR drops to 24.0%-33.0%. Activating +DTN or +SEC +ADAPT +DTN restores unicast SAR coordination to 73.5%-77.5% PDR for DSDV and 66% PDR for AODV, proving that DTN architecture is essential for tactical unicast traffic across radio dead zones.

Control Overhead and Overlay Relay Transmissions

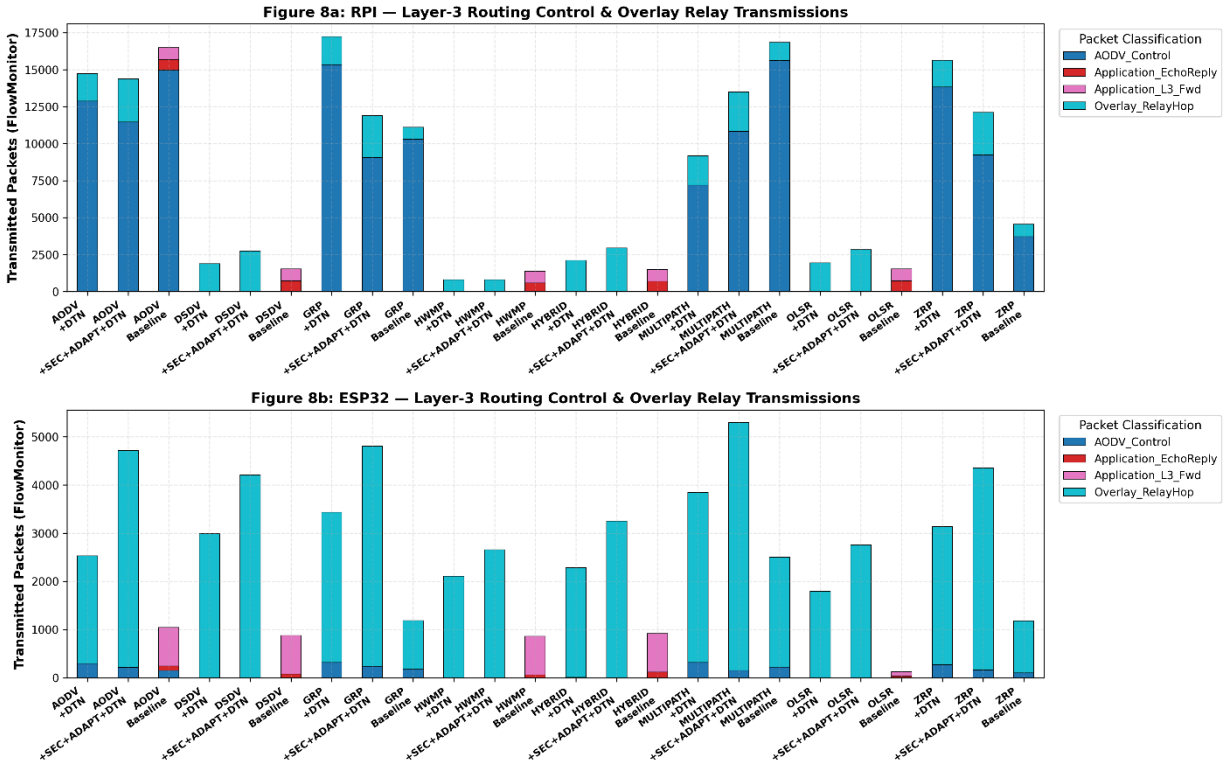


Figure 8.8: Layer-3 Routing Control & Overlay Relay Transmissions

Detailed Analysis

In Figure 8.8, Layer-3 Control Overhead vs. Application Data: Proactive protocols (OLSR, DSDV) generate continuous background control frames (OLSR_Control on UDP Port 698, DSDV_Control on UDP Port 269) regardless of traffic demand. Reactive protocols (AODV)

generate control spikes (AODV_Control on UDP Port 654) only during initial path discovery and on link-breaking and recovery of broken links.

Overlay Relay Overhead in DTN Modes: Modes with +DTN enabled exhibit taller Overlay_RelayHop bars (UDP Port 9999) compared to pure MANET modes. This increased overhead represents DTN binary replication (L=16 tickets on RPI and L=6 on ESP32), periodic 1-hop Hello beacons, and Watchdog acknowledgements (PKT_WATCHDOG_CUSTODY).

CONCLUSION AND FUTURE SCOPE

Conclusion

This project demonstrated the implementation, design and evaluation of a multi-protocol ad-hoc mesh framework designed in Network Simulator 3 (NS-3.48). Designed specifically for post-disaster scenarios where centralized network infrastructure has failed, the system provides off-grid communication for both civilian disaster victims (SOS Anycast) and first-responder's field communication (SAR Unicast).

The main empirical findings can be summarized as follows:

1. Physical hardware limitations determine protocol feasibility: while routing on high-performance single board computers, such as Raspberry Pi 4/5, preserves high connectivity, routing on low-power microcontrollers, such as ESP32, results in total failure of basic MANET protocols owing to multiple path breaks.
2. Disruption Tolerance becomes necessary for embedded microcontrollers: the Binary Spray & Wait Delay-Tolerant Networking (DTN) application overlay restores packets' survivability in ESP32 networks.
3. Spatial Diversity Advantage of SOS Anycast: according to the traffic analysis, Victim SOS Anycast (Flows 1-4) demonstrates significantly higher success rate than SAR Unicast coordination (Flows 5-6), since any SAR node can satisfy the emergency SOS request.
4. Zero-Configuration Architecture: with the help of FNV-1a 16-bit MAC-to-IPv4 hashing, the system proves that fully autonomous zero-configuration mesh network can work without any DHCP servers.

Along with the simulation framework, the project implements MeshSOS - an android application created in Kotlin. This application realizes offline SOS communication using Wi-Fi Aware and Bluetooth Classic at the application layer, including messaging, ACKs processing, and messages storing.

Future Scope

RL for Dynamic Adaptive DTN Ticket Allocation: By replacing static binary ticket split (L=16 or L=6) with RL, particularly, lightweight Q-learning algorithms, nodes would be able to make adaptive ticket split decisions in accordance with current mobility speeds and historical values of queues occupancy and contact probabilities.

Further Development of Mesh Network Using Android Application MeshSOS: The application MeshSOS can become a powerful emergency communication tool in future by adding additional features such as dynamic peer discovery, advanced multi-hop routing, background connectivity, location sharing, real-time tracking of responders, emergency group

communication, multimedia evidence exchange and others. The offline map function can be added to the application to let users know about available nearby responders and the affected area. Additionally, some optimization can be performed to decrease battery consumption and provide communication even after disconnection from the network for a certain period of time.

Expansion of the Framework to Physical Devices Based on ESP32 and LoRa Modules: The framework can be expanded to physical embedded devices such as ESP32 microcontrollers combined with LoRa modules. Long range and low power consumption can be provided with the use of LoRa technology. That is why LoRa can be used as a long range technology in disaster scenarios when mobile networks and Wi-Fi connection cannot be used. ESP32 microcontrollers combined with LoRa modules can become dedicated mesh communication nodes which would allow transferring SOS messages over large geographical distances.

REFERENCES

[1] ns-3 Project, "ns-3 Model Library, Release 3.48," 2026. [Online]. Available: https://www.nsnam.org/docs/release/3.48/models/ns-3-model-library.pdf. [Accessed October 2026].

[2] L. Campanile, M. Gribaudo, M. Iacono, F. Marulli and M. Mastroianni, "Computer Network Simulation with ns-3: A Systematic Literature Review," *Electronics,* vol. 9, no. 2, p. 272, 2020.

[3] B. S. Manoj and A. H. Baker, "Communication Challenges in Emergency Response," *Communications of the ACM,* vol. 50, no. 3, p. 51–53, March 2007.

[4] I. F. Akyildiz, X. Wang and W. Wang, "Wireless Mesh Networks: A Survey," *Computer Networks,* vol. 47, no. 4, p. 445–487, March 2005.

[5] Android Developers, "Wi-Fi Aware overview," [Online]. Available: https://developer.android.com/develop/connectivity/wifi/wifi-aware. [Accessed 29 October 2026].

[6] C. E. Perkins and E. M. Royer, "Ad hoc On-Demand Distance Vector Routing," in *Proceedings of the Second IEEE Workshop on Mobile Computing Systems and Applications (WMCSA '99)*, New Orleans, LA, USA, 1999.

[7] D. B. Johnson and D. A. Maltz, "Dynamic Source Routing in Ad Hoc Wireless Networks," in *Mobile Computing*, Norwell, MA, USA, Kluwer Academic Publishers, 1996, p. 153–181.

[8] T. Clausen and P. Jacquet, "Optimized Link State Routing Protocol (OLSR)," Internet Engineering Task Force (IETF), 2003.

[9] C. E. Perkins and P. Bhagwat, "Highly Dynamic Destination-Sequenced Distance-Vector Routing (DSDV) for Mobile Computers," in *Proceedings of the Conference on Communications Architectures, Protocols and Applications (SIGCOMM '94)*, London, UK, 1994.

[10] G. R. Hiertz, T. J. J. Denteneer, S. Max, R. Taori, J. Cardona, L. Berlemann and B. Walke, "IEEE 802.11s: The WLAN Mesh Standard," *IEEE Wireless Communications,* vol. 17, no. 1, p. 104–111, February 2010.

[11] J. Li, J. Jannotti, D. S. J. D. Couto, D. R. Karger and R. Morris, "A Scalable Location Service for Geographic Ad Hoc Routing," in *Proceedings of the 6th ACM International Conference on Mobile Computing and Networking (MobiCom '00)*, Boston, MA, USA, 2000.

[12] B. Karp and H. T. Kung, "GPSR: Greedy Perimeter Stateless Routing for Wireless Networks," in *Proceedings of the 6th Annual ACM/IEEE International Conference on Mobile Computing and Networking (MobiCom 2000)*, Boston, MA, USA, 2000.

[13] Z. J. Haas and M. R. Pearlman, "The Zone Routing Protocol (ZRP) for Ad Hoc Networks," 1998. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-ietf-manet-zone-zrp-01.txt. [Accessed October 2026].

[14] T. Spyropoulos, K. Psounis and C. S. Raghavendra, "Spray and Wait: An Efficient Routing Scheme for Intermittently Connected Mobile Networks," in *Proceedings of the 2005 ACM SIGCOMM Workshop on Delay-tolerant Networking (WDTN '05)*, Philadelphia, PA, USA, 2005.

[15] S. Marti, T. J. Giuli, K. Lai and M. Baker, "Mitigating Routing Misbehavior in Mobile Ad Hoc Networks," in *Proceedings of the 6th Annual International Conference on Mobile Computing and Networking (MobiCom '00)*, Boston, MA, USA, 2000.

[16] S. Cheshire, B. Aboba and E. Guttman, "Dynamic Configuration of IPv4 Link-Local Addresses (RFC 3927)," Internet Engineering Task Force (IETF), 2005.

[17] T. Camp, J. Boleng and V. Davies, "A Survey of Mobility Models for Ad Hoc Network Research," *Wireless Communications and Mobile Computing,* vol. 2, no. 5, p. 483–502, 2002.

[18] Espressif Systems, "ESP32 Series Datasheet, Version 5.3," July 2026. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.html. [Accessed 30 October 2026].

[19] K. R. Fall, "A Delay-Tolerant Network Architecture for Challenged Internets," in *Proceedings of the 2003 ACM SIGCOMM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, Karlsruhe, Germany, 2003.

[20] V. Cerf, S. Burleigh, A. Hooke, L. Torgerson, R. Durst, K. Scott, K. Fall and H. Weiss, "Delay-Tolerant Networking Architecture," RFC Editor, 2007.

[21] M. Bahr, "Update on the Hybrid Wireless Mesh Protocol of IEEE 802.11s," in *Proceedings of the IEEE 4th International Conference on Mobile Ad Hoc and Sensor Systems (MASS 2007)*, Pisa, Italy, 2007.

[22] R. D. Todakar, B. H. Gavali, K. M. Y. Shaikh and J. M. Patil, "AI-Driven, Self-Recovering Communication Network for Deep Disaster Zone," *International Journal of Advanced Research in Science, Communication and Technology,* vol. 5, no. 1, p. 499–513, October 2025.

[23] Android Developers, "Bluetooth overview," [Online]. Available: https://developer.android.com/develop/connectivity/bluetooth. [Accessed 29 October 2026].

[24] M. S. Corson and J. P. Macker, "Mobile Ad hoc Networking (MANET): Routing Protocol Performance Issues and Evaluation Considerations," 1999. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2501. [Accessed 29 October 2026].

[25] A. Vahdat and D. Becker, "Epidemic Routing for Partially-Connected Ad Hoc Networks," Duke University, Durham, NC, USA, 2000.

[26] A. Lindgren, A. Doria and O. Schelén, "Probabilistic Routing in Intermittently Connected Networks," *ACM SIGMOBILE Mobile Computing and Communications Review,* vol. 7, no. 3, p. 19–20, July 2003.

[27] Raspberry Pi Ltd, "Raspberry Pi computer hardware," [Online]. Available: https://www.raspberrypi.com/documentation/computers/raspberry-pi.html. [Accessed October 2026].

[28] B. Karp and H. T. Kung, "GPSR: Greedy Perimeter Stateless Routing for Wireless Networks," in *Proceedings of the 6th Annual International Conference on Mobile Computing and Networking (MobiCom '00)*, Boston, MA, USA, 2000.

[29] Semtech Corporation, "SX1261/2 Data Sheet (Rev. 2.1)," December 2021. [Online]. Available: https://uelectronics.com/wp-content/uploads/2022/12/Datasheet-LoRa-SX1262.pdf. [Accessed October 2026].

[30] H. ElSawy, A. Zhaikhan, M. A. Kishk and M.-S. Alouini, "A Tutorial-Cum-Survey on Percolation Theory With Applications in Large-Scale Wireless Networks," *IEEE Communications Surveys & Tutorials,* vol. 26, no. 1, p. 428–460, 2024.

[31] O. Dousse, F. Baccelli and P. Thiran, "Impact of Interferences on Connectivity in Ad Hoc Networks," *IEEE/ACM Transactions on Networking,* vol. 13, no. 2, p. 425–436, 2005.

Yajur chatnani

peer

Minor Project

B.Tech Minor

Indian Institute of Information Technology BP

Document Details

Submission ID

trn:oid:::1:3696800523

Submission Date

Oct 31, 2026, 8:08 PM GMT+5:30

Download Date

Oct 31, 2026, 8:12 PM GMT+5:30

File Name

24U020062_and_24U020056final.pdf

File Size

3.0 MB

66 Pages

14,276 Words

81,164 Characters

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **94** Not Cited or Quoted 13%
  Matches with neither in-text citation nor quotation marks
- **11** Missing Quotations 1%
  Matches that are still very similar to source material
- **6** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

11% Internet sources
7% Publications
9% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Yajur chatnani

peer

Minor Project

B.Tech Minor

Indian Institute of Information Technology BP

Document Details

Submission ID

trn:oid:::1:3696800523

Submission Date

Oct 31, 2026, 8:08 PM GMT+5:30

Download Date

Oct 31, 2026, 8:11 PM GMT+5:30

File Name

24U020062_and_24U020056final.pdf

File Size

3.0 MB

66 Pages

14,276 Words

81,164 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.